Blue Cloud Softech Solutions Ltd.
From Information to *Intelligence*

BLUE CLOUD SOFTECH SOLUTIONS LTD. | SNIST INTERNSHIP CAPSTONE

Building a Zero Trust / SASE Architecture on Cloudflare's Free Tier

Consolidated Capstone Report — All Seven Layers

Track	15-Day Hands-On SASE / Zero Trust Capstone
Platform	Cloudflare Zero Trust (Cloudflare One) — Free Plan, up to 50 users
Cloudflare Team Domain	flat-river-d677.cloudflareaccess.com
Identity Provider	Microsoft Entra ID (namespace: demotest.space)
Program	SNIST Internship Cohort — BCSS Academia

Generated from the team's consolidated build report, structured and formatted against the mentor-issued Capstone Brief and Team Step-by-Step Guide.

Program Roster

Layer	Team Focus	Students
1	Identity	Maddiguntla Balaji Bhargav, Dhan Sri Sai Suraj Singh
2	Device	Veera Renitha Lakshmi, P Akshay
3	Network (ZTNA)	Gunnala Kavya, Nomula Snehith
4	Application	Duvvapu Jahnav Anand Patel, Benduri Akshaya
5	Web/Internet (SWG)	Vilasagarapu Rushikesh, Gunti Naveen Reddy
6	Data (DLP)	Nunna Aahlada, Shaik Azahruddin
7	Visibility (DEX)	Kankati Abhigna, Mahi Mahir

Alignment with the Mentor's Capstone Brief

The consolidated report below was checked section-by-section against the mentor-issued *Capstone Project Brief* (7-layer SASE architecture, 15-day execution plan, deliverables checklist and evaluation rubric) and the *Team Step-by-Step Guide*. Coverage against the Brief's seven required layers is as follows:

#	Brief Layer	Cloudflare Component (Free Tier)	Documented in this Report
1	Identity	Microsoft Entra ID (external IdP) via Cloudflare One	Team 1, p. below
2	Device	Cloudflare WARP client enrollment	Team 2
3	Network (ZTNA)	WARP Connector / Cloudflare Tunnel + Private Network routes	Team 3 (plus a supplementary Sophos/Azure VPN deep-dive)
4	Application	Cloudflare Access self-hosted application + policy	Team 4
5	Web/Internet (SWG)	Gateway DNS & HTTP filtering policies	Team 5
6	Data (DLP)	Gateway DLP — predefined profile	Team 6
7	Visibility (DEX)	Digital Experience Monitoring + Network Analytics	Team 7

> All seven layers named in the Brief are represented. Team 3's section goes beyond the Brief's core ZTNA scope by including a supplementary Sophos Firewall / Azure / IPsec VPN write-up; this is kept as an appendix-style deep-dive within the Team 3 section since it documents real work completed but sits outside the Cloudflare-only architecture the Brief specifies.

Table of Contents

Team 1 — Identity Layer Documentation

1. Introduction: Zero Trust and SASE
2. Team 1 — Identity Layer
3. Cloudflare One Identity Configuration
4. Cloudflare One Team Configuration
5. Identity Authentication Flow
6. Identity Verification
7. Role of the Identity Layer in the Overall Project
8. Configuration Summary
9. Conclusion

Team 2 — DEVICE

Introduction – Device Layer in Zero Trust / SASE
Project Objective and What We Built
Device Enrollment Permissions
Installing Cloudflare One Client (WARP)
Enrolling the Device
Verifying the Enrolled Device
Device Enrollment Policy Testing
Allowed and Blocked Users
Implementation Results
Issues Encountered and Resolution
Conclusion

Team 3 — NETWORK (ZTNA)

1. Introduction
2. Objective
3. Basic Architecture
4. Components Used
5. Step 1 — Set Up the Internal Test Server
1. Introduction
2. Azure Infrastructure
3. Sophos Firewall Deployment
4. Sophos Central Management
5. Microsoft Entra ID Authentication
6. Microsoft Entra Group Integration
7. Remote Access IPsec VPN
8. Network Security Configuration
9. Sophos Connect Client
10. Testing Results

Team 4 — APPLICATION

1. Objective
2. Implementation Flow
3. Step-by-Step Procedure
4. Final Verification / Completion Criteria
5. Result

Team 5 — WEB / INTERNET (SWG)

1. Introduction
2. Objective

3. Evidence Selection
4. Step-by-Step Implementation and Progress
5. Implementation Results
6. Evidence Notes and Limitations
7. Conclusion

Team 6 — Data Loss Prevention (DLP)

1. Objective
2. Step 1 — Enable HTTPS Traffic Inspection
3. Step 2 — Configure the DLP Profile
4. Step 3 — Create the HTTP Gateway DLP Policy
5. Step 4 — Test and Trigger the Policy
6. Step 5 — Gather Evidence and Analytics
7. Summary

Team 7 — Visibility: DEX and Network Flow Monitoring

1. Objective
2. Scope
3. Prerequisites
4. Test 1 – HTTP Synthetic Monitoring
5. Test 2 – Network Latency Monitoring
6. Network Analytics and Flow Monitoring
7. Overall Findings
8. Conclusion
9. Reference

Team 1 — Identity Layer Documentation

Zero Trust / SASE Capstone — Cloudflare Free Tier

Blue Cloud Softech Solutions Ltd. | SNIST Internship Capstone

Scope. This document covers the work of Team 1, the Identity Layer: Microsoft Entra ID as the Identity Provider (IdP) and its integration with Cloudflare One. Device-related work belongs to Team 2 and is documented separately.

1. Introduction: Zero Trust and SASE

What is Zero Trust?

Traditional security works like a castle: build a strong wall, such as a firewall or VPN, and anyone inside is trusted. The weakness is that an attacker who gets past the wall once, for example with stolen VPN credentials or a compromised laptop, may be able to move freely.

Zero Trust replaces this with the principle of “never trust, always verify.” Access decisions are based on verified identity, device status and relevant context, not on whether a user happens to be on an internal network. In this project, the Identity Layer establishes who the user is before any downstream Zero Trust control makes an access decision.

What is SASE?

Secure Access Service Edge (SASE) combines Zero Trust security functions with wide-area networking capabilities and delivers them through cloud infrastructure. Here, Cloudflare One (Cloudflare Zero Trust) is the security platform and Microsoft Entra ID is the external identity source. Cloudflare consumes the identity information provided through Entra and uses it in the access-control process.

2. Team 1 — Identity Layer

The Identity Layer is the first control point in the architecture. Its purpose is to authenticate users and establish a trusted identity before access is granted to protected resources. The main elements are:

- **Identity Provider (IdP):** Microsoft Entra ID, the source of truth for project identities.
- **User accounts:** test/project accounts provisioned in the Microsoft Entra tenant.
- **Identity integration:** Cloudflare One is connected to Microsoft Entra ID so that users can authenticate through the configured IdP.
- **Identity namespace:** project identities use the demotest.space domain (@demotest.space).

Microsoft Entra ID as the Identity Provider

Project users were created and managed in the Microsoft Entra tenant, which gives Cloudflare One a single identity source to rely on during authentication. Using demotest.space for all project identities keeps the namespace consistent, so the Cloudflare environment works with the same user identities that exist in the Entra tenant.

3. Cloudflare One Identity Configuration

The Cloudflare One environment is set up with the team name flat-river-d677 and the team domain flat-river-d677.cloudflareaccess.com, with Microsoft Entra ID as its external IdP. This lets Cloudflare One authenticate users through the configured IdP and use the resulting identity information for Zero Trust access decisions. Figure 1 shows the Cloudflare One Client profile with the team name.

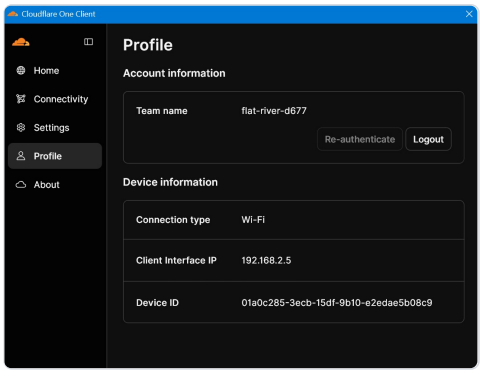


fig1.png

Figure 1. Cloudflare One Client profile showing the configured Cloudflare One team name and client connection information.

The device information in this screenshot (connection type, interface IP and device ID) is part of the Device Layer and is not covered by this document.

4. Cloudflare One Team Configuration

The Cloudflare One dashboard is the management layer for the Zero Trust environment. Its overview lists the team domain, team name, users, applications and device-related activity, while Microsoft Entra ID remains the external IdP. The team details relevant to the Identity Layer appear in the Account details panel of Figure 2.

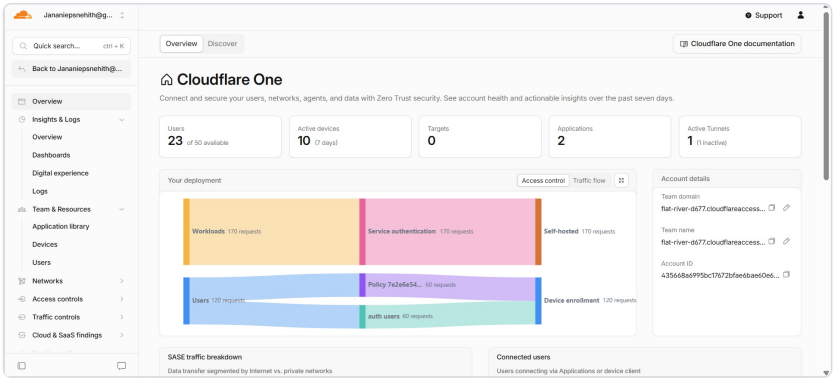


fig2.png

Figure 2. Cloudflare One dashboard showing the configured Zero Trust environment and team information.

The device enrollment area visible in the dashboard belongs to the Device Layer (Team 2) and is outside the scope of Team 1.

5. Identity Authentication Flow

The authentication flow for the Identity Layer works as follows:

1. A project user attempts to access the Cloudflare Zero Trust environment.
2. Cloudflare One uses the configured external Identity Provider.
3. The user is authenticated through Microsoft Entra ID.
4. Entra ID provides the identity information required for the authentication flow.
5. Cloudflare uses the authenticated identity for subsequent Zero Trust access decisions.

User → **Cloudflare** → Microsoft → **Authentication** → **Verified** → Zero Trust
One** Entra ID** Identity** Access
Decision**

Figure 3. Identity authentication flow.

6. Identity Verification

The Identity Layer was validated by confirming that the Cloudflare environment is connected to the configured identity system and that project identities are associated with the demotest.space namespace. The purpose is to ensure that the downstream Zero Trust components work from the same identity source. Device enrollment and Gateway filtering are outside the scope of this validation.

7. Role of the Identity Layer in the Overall Project

The Identity Layer provides the identity foundation for the remaining layers. Team 1 establishes the identity source through Microsoft Entra ID and its integration with Cloudflare One; the other teams build on it.

Team	How it builds on the Identity Layer
Team 2	Uses the established identity for device-related controls and WARP enrollment
Team 3	Builds network and route access controls on top of the established identity
Team 4	Uses the identity and access controls when working with the published application

8. Configuration Summary

Component	Configuration
Identity Provider	Microsoft Entra ID
Project Domain	demotest.space
Cloudflare Platform	Cloudflare One / Cloudflare Zero Trust
Cloudflare Team Name	flat-river-d677
Cloudflare Team Domain	flat-river-d677.cloudflareaccess.com
Team Responsibility	Identity Layer
Primary Purpose	User authentication and identity verification

9. Conclusion

The Identity Layer establishes the identity foundation of the project’s Zero Trust/SASE architecture. Microsoft Entra ID serves as the external Identity Provider, and Cloudflare One is the security platform that consumes this identity and uses it for access decisions.

The configured Cloudflare One team, flat-river-d677, together with the demotest.space namespace, provides a consistent environment for authentication and for the controls that follow. With a centralized identity source, later layers can base access decisions on a known, authenticated user rather than on network location alone.

Team 2 — DEVICE

Introduction – Device Layer in Zero Trust / SASE

Cloudflare WARP Device Enrollment

Field	Details
Project	Building a Zero Trust / SASE Architecture on Cloudflare's Free Tier
Team	Team 2 - Device
Students	Veera Renitha Lakshmi, P Akshay
Platform	Cloudflare Zero Trust + Cloudflare One Client (WARP)

Project Objective and What We Built

Objective

The objective of Team 2 is to secure and manage endpoint devices using **Cloudflare WARP**.

The team configures a device enrollment policy so that only authorized users can enroll their devices into the Cloudflare Zero Trust organization. After enrollment, the devices are verified in the Zero Trust dashboard.

The Team 2 guide describes WARP as the device-side “badge” that allows an endpoint to participate in the protected environment.

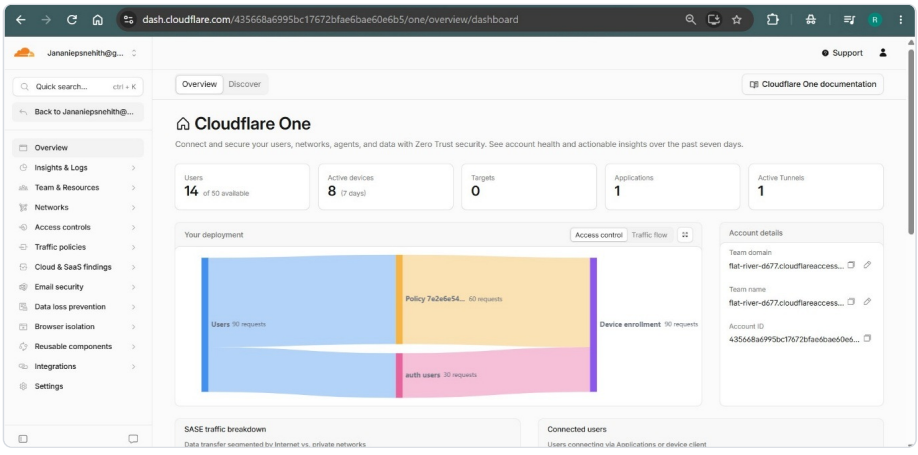
What We Built

We worked on the **Device layer** of the Zero Trust / SASE architecture. Our work included:

- Configuring device enrollment permissions
- Restricting enrollment based on the organization's email domain
- Installing Cloudflare One Client (WARP)
- Enrolling an authorized device
- Verifying the enrolled device in the Cloudflare dashboard
- Testing the enrollment policy with allowed and blocked users

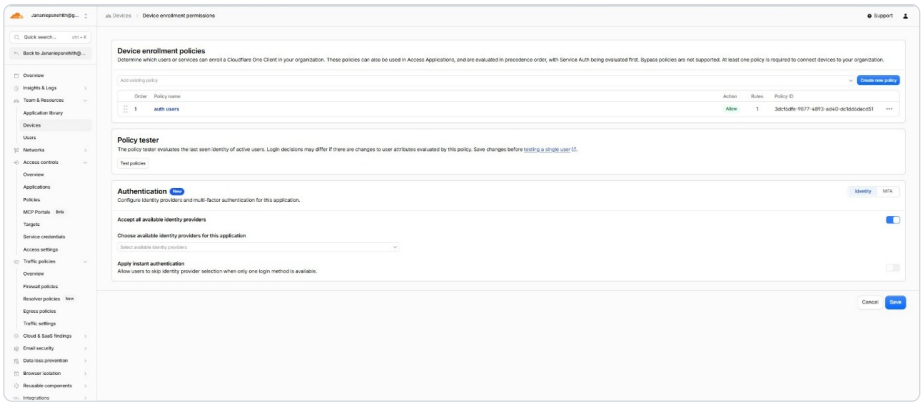
Device Enrollment Permissions

First, open the **Cloudflare Zero Trust Dashboard**.



Navigate to:

Team & Resources → Devices → Device profiles → Management → Device enrollment → Device enrollment permissions



Configuration used in our project

Policy: auth users

Action: Allow

Include: Emails ending in @demotest.space

This means users with an email address belonging to the approved @demotest.space domain are allowed to enroll their devices.

Installing Cloudflare One Client (WARP)

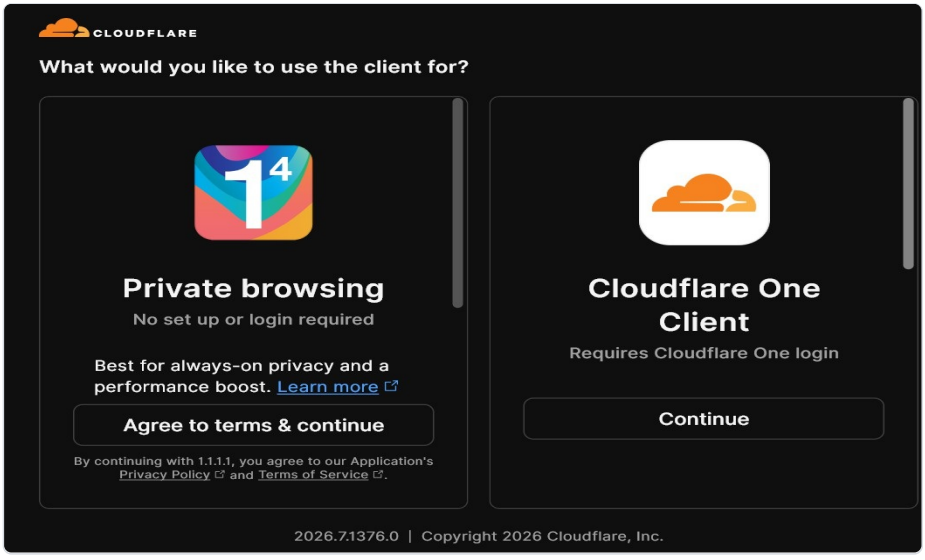
Download the Cloudflare WARP client from:

https://1.1.1.1/

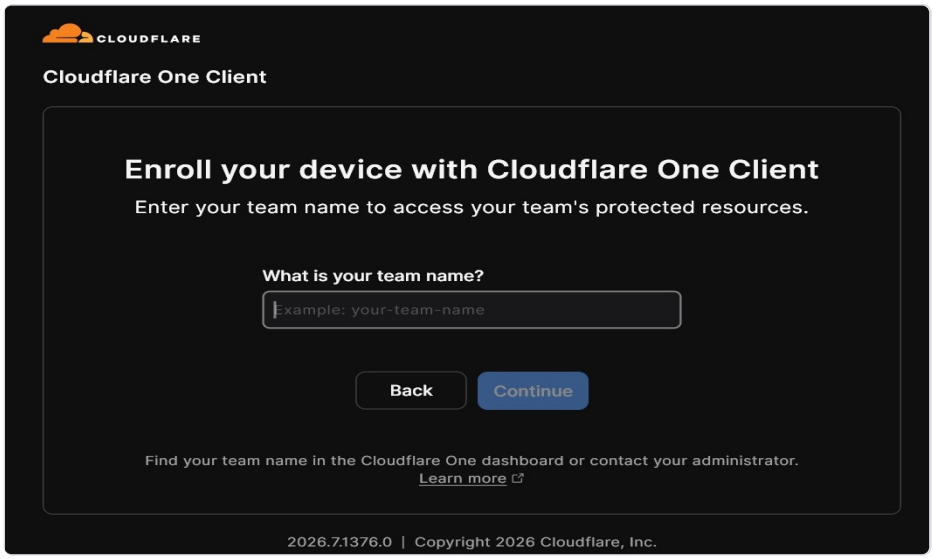
Install the version suitable for the operating system.

Enrolling the Device

Open **Cloudflare One Client**.



Select: **Login with Cloudflare Zero Trust**

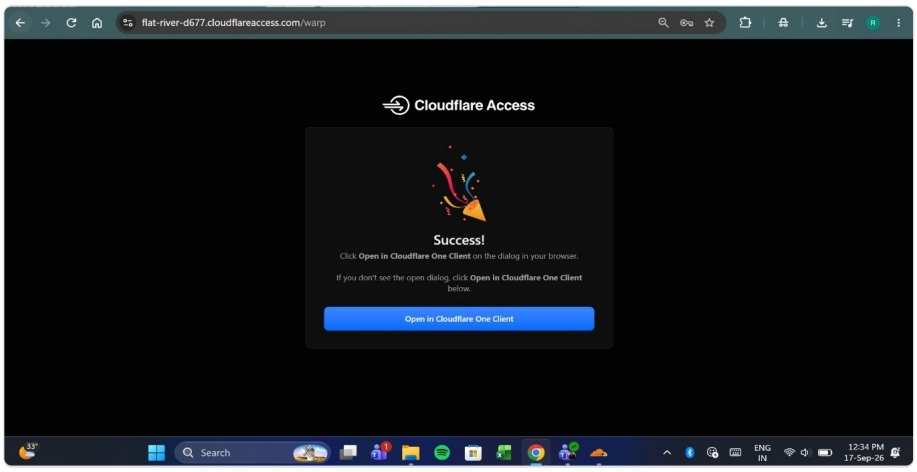


Enter the team name: **Flat-river-d677**

The browser then opens the Cloudflare authentication page.

Sign in using an authorized **@demotest.space** account.

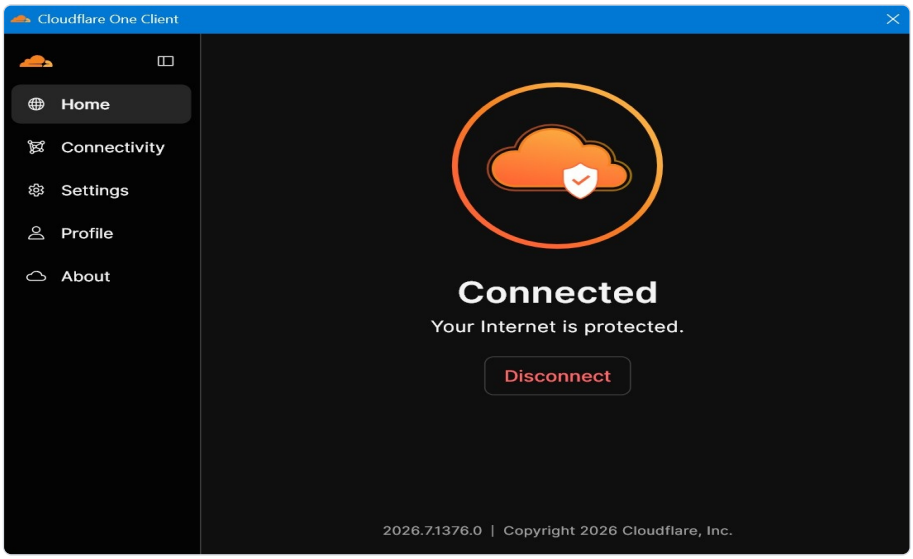
After successful authentication, Cloudflare displays the **Success** page.



Click:

Open in Cloudflare One Client

The client should then become connected.



Verifying the Enrolled Device

After successful enrollment, return to the Cloudflare Zero Trust Dashboard. Navigate to:

Team & Resources → Devices → Your devices

The enrolled device should appear in the device list.

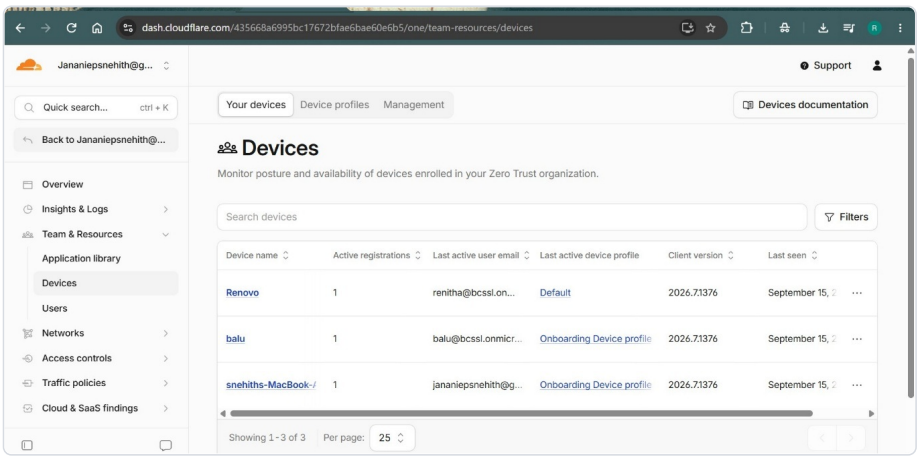
Our enrolled device

Device: Renovo

User: renitha@demotest.space

Active registration: 1

This confirms that the endpoint has been successfully registered with the Cloudflare Zero Trust organization.



Device Enrollment Policy Testing

The device enrollment policy was tested using the **Policy Tester**.

The policy tester evaluates users against the configured policy and shows whether they are **allowed** or **blocked**.

Allowed and Blocked Users

Allowed users

The policy tester showed users with the approved @demotest.space domain as:

Allowed

Examples shown:

- renitha@demotest.space — Allowed
- ncs@demotest.space — Allowed
- balu@demotest.space — Allowed

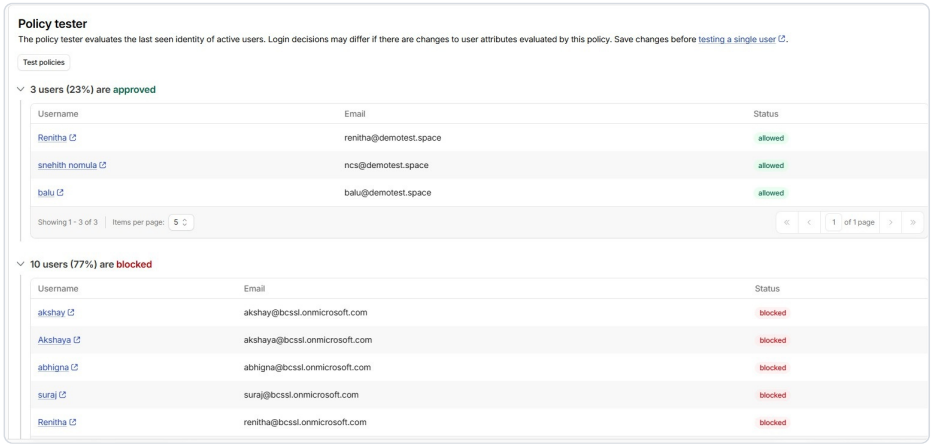
Blocked users

Users outside the approved domain were shown as:

Blocked

Examples included users with: bcssl.onmicrosoft.com

This demonstrates that the email-domain restriction is working as intended.



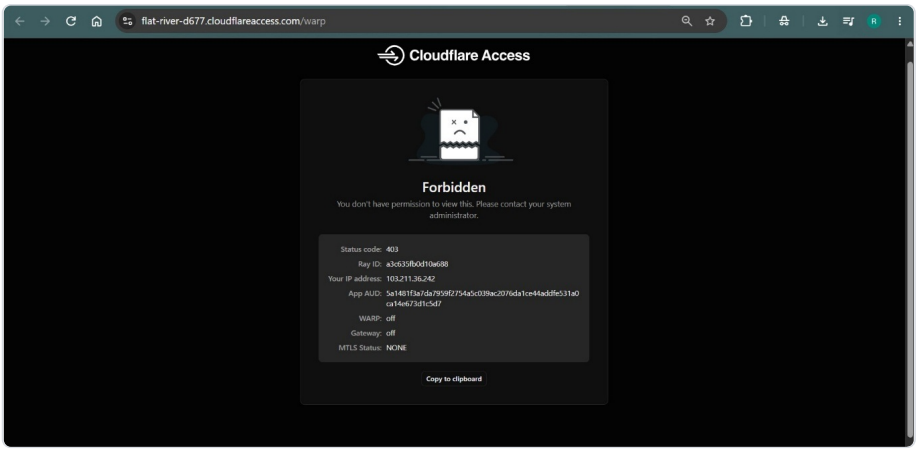
Implementation Results

The following results were achieved:

Test	Result
Device enrollment policy configured	Successful
Authorized @demotest.space user	Allowed
Unauthorized domain users	Blocked

Test	Result
WARP installation	Successful
Device authentication	Successful
Device registration	Successful
Device visible in dashboard	Successful

Issues Encountered and Resolution



During testing, an authentication attempt initially resulted in a **403 Forbidden** page because the WARP status was shown as **off**.

After completing the correct Cloudflare Zero Trust enrollment flow, authentication was successful and the device was registered.

Conclusion

Team 2 successfully implemented the **Device layer** using Cloudflare One Client (WARP).

The device enrollment policy was configured to allow users from the approved organization domain. An authorized device was successfully enrolled and verified in the Cloudflare Zero Trust Devices dashboard. Policy testing also showed that users from the approved domain were allowed while users outside the domain were blocked.

This demonstrates how Cloudflare WARP can be used as part of a Zero Trust architecture to control and manage endpoint device enrollment.

Team 3 — NETWORK (ZTNA)

Private Network Access Using Cloudflare Zero Trust

Students: Gunnala Kavya, Nomula Snehith
Project: BCSS Academia - SNIST Capstone
Team 3: Network (ZTNA)

1. Introduction

What is the project?

The objective of Team 3 is to implement **Zero Trust Network Access (ZTNA)** for a private internal resource. Instead of exposing an entire internal network to users through a traditional VPN, the project provides controlled access to **one specific private resource**. The private resource is made accessible through a secure Cloudflare tunnel, and access is restricted to authorized users/devices.

The basic concept is:

Authorized User

↓

Enrolled Device

↓

WARP

↓

Cloudflare Zero Trust

↓

Access Policy

↓

Private Network Route

↓

Cloudflare Tunnel

↓

Internal Test Server

The project therefore follows the principle of: Allow access to a specific resource rather than providing unrestricted access to the entire internal network.

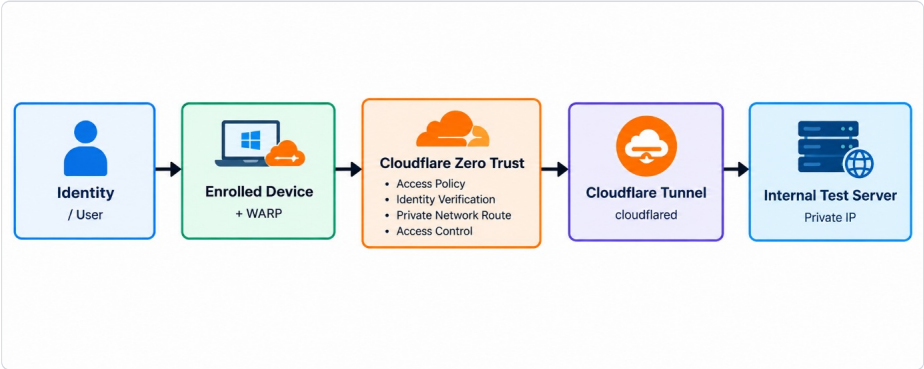
2. Objective

The main objectives of Team 3 are:

1. Set up an internal test server as the protected private resource.
2. Create a Cloudflare Tunnel using the cloudflared connector.
3. Connect the internal environment to Cloudflare.
4. Configure a private network route to the test server.
5. Create a **Private Network** application.
6. Configure an Access Policy for the authorized identity group.
7. Access the private server from an enrolled device with WARP enabled.
8. Verify that the same server becomes unreachable when WARP is disabled.
9. Demonstrate that access is controlled by the Zero Trust configuration.

3. Basic Architecture

The implemented architecture can be represented as follows:



The user first uses an enrolled device with WARP enabled. The request reaches Cloudflare Zero Trust, where the configured access controls determine whether the user is allowed to access the private resource. If the request is permitted, traffic is sent through the configured private route and Cloudflare Tunnel toward the internal test server.

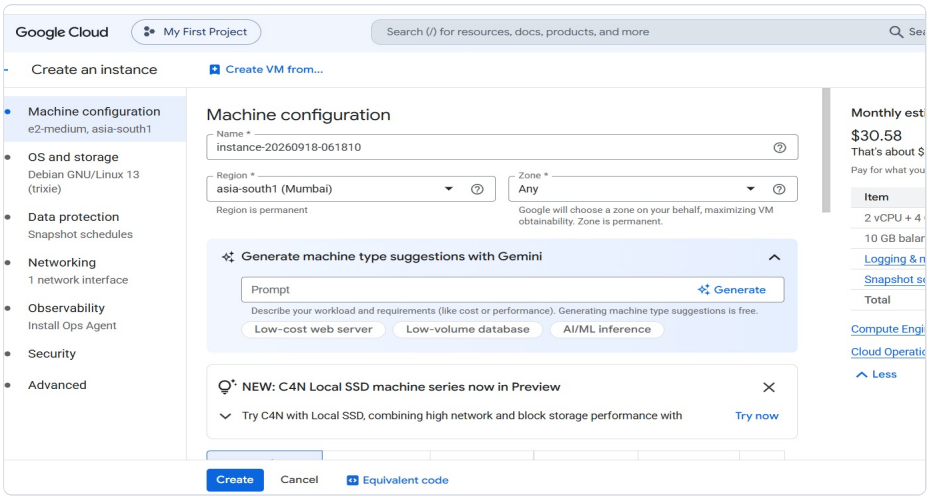
4. Components Used

Component	Purpose
Cloudflare Zero Trust	Central platform for implementing ZTNA
Cloudflare Tunnel	Provides connectivity between Cloudflare and the internal environment
cloudflared	Connector running on the Linux machine
Linux machine/VM	Hosts the connector and provides access to the internal environment
Internal Test Server	Private resource being protected
Private Network Route	Specifies the private IP/subnet that should be reached through the tunnel
Private Network Application	Defines the private resource in the Zero Trust configuration
Access Policy	Controls which users/groups can access the resource
WARP	Client on the enrolled device used to establish the user's connection to the Zero Trust environment
Microsoft Entra ID	User/identity environment used in the project, as configured by the project teams

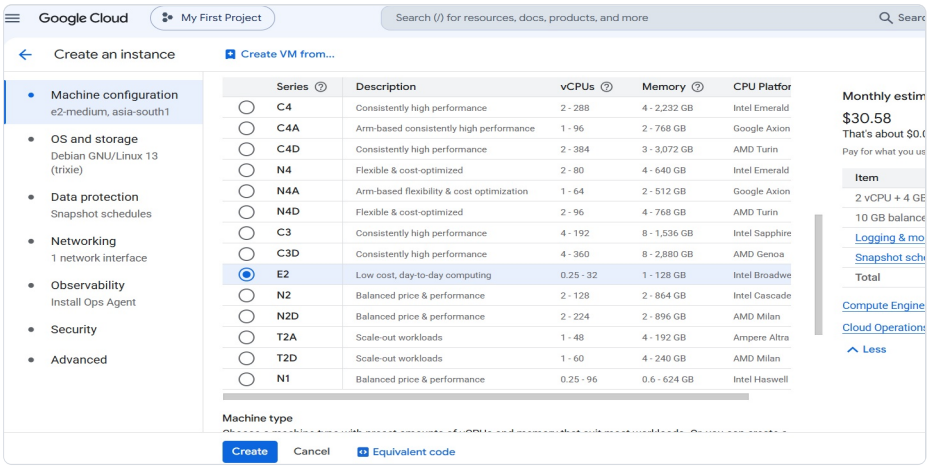
5. Step 1 — Set Up the Internal Test Server

The first step was to select or create an internal test server. The test server acts as the **private resource** that Team 3 needs to protect. It can be a simple web server, Wazuh dashboard, or basic webpage as specified in the project instructions. But here we are using the simple web server created with python code.

In GCP we have to create the VM instance. Open Google cloud and go to VM instances -> Create new instance -> Machine configuration.



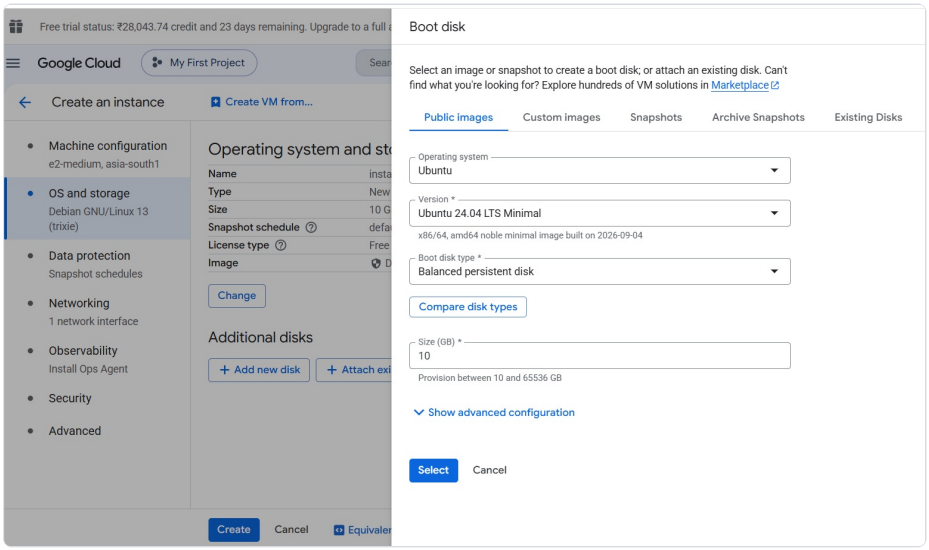
Machine type: E2 and fill all the detail then click on create.



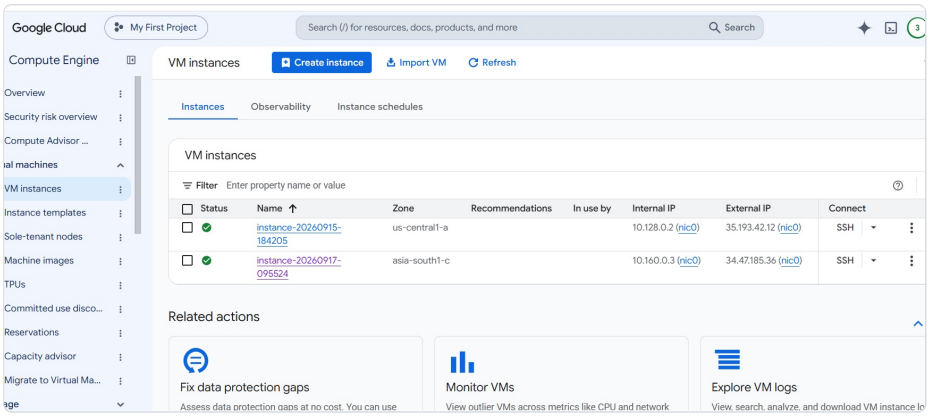
Select OS and storage click on change.

Select OS: Ubuntu

Version: Ubuntu 24.04 LTS Minimal then click on select



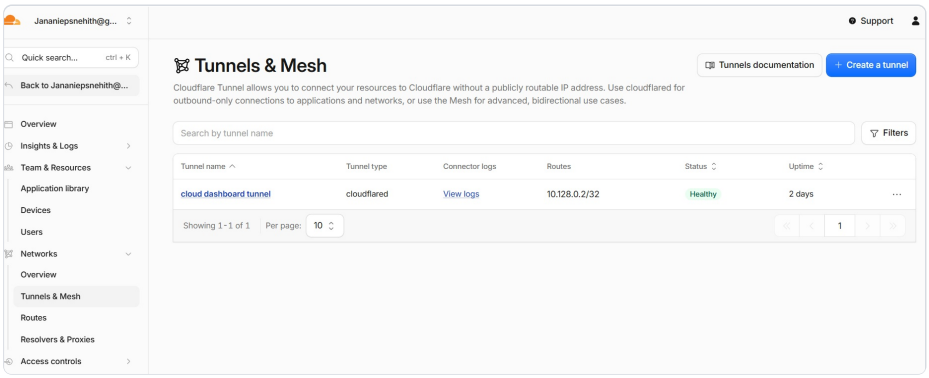
Instance created successfully.

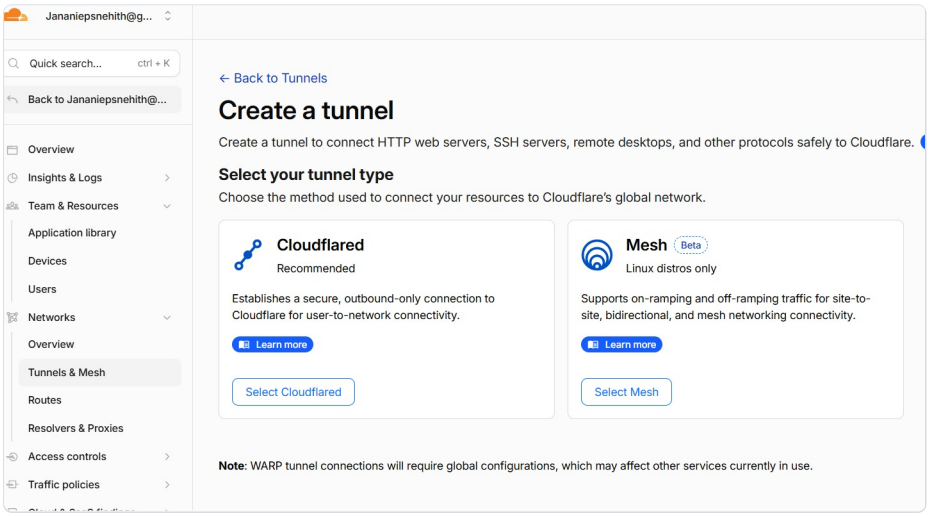


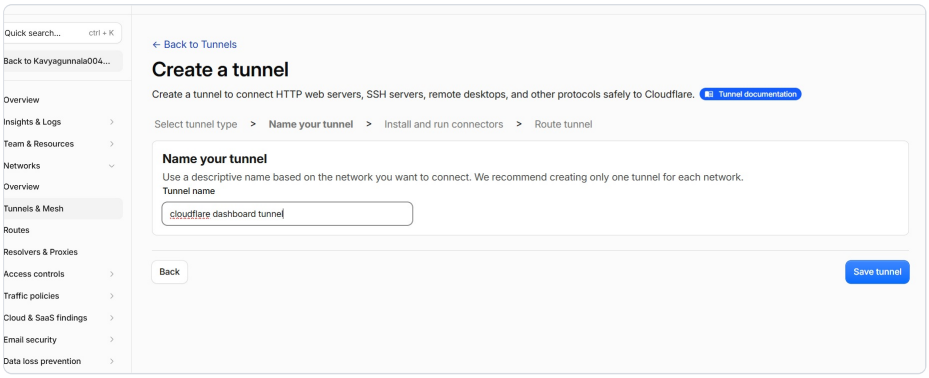
Step 2 - Create the Cloudflare Tunnel

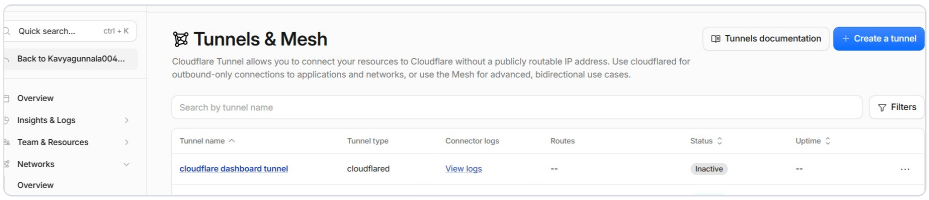
After preparing the internal server, the next step was to create a Cloudflare Tunnel. A new tunnel was created using the **Cloudflared connector** option. The purpose of this tunnel is to provide a secure connection between the Cloudflare network and the internal environment.

Navigate to: **Cloudflare Zero Trust → Networks → Tunnels & Mesh → Create a tunnel → Select cloudfared**

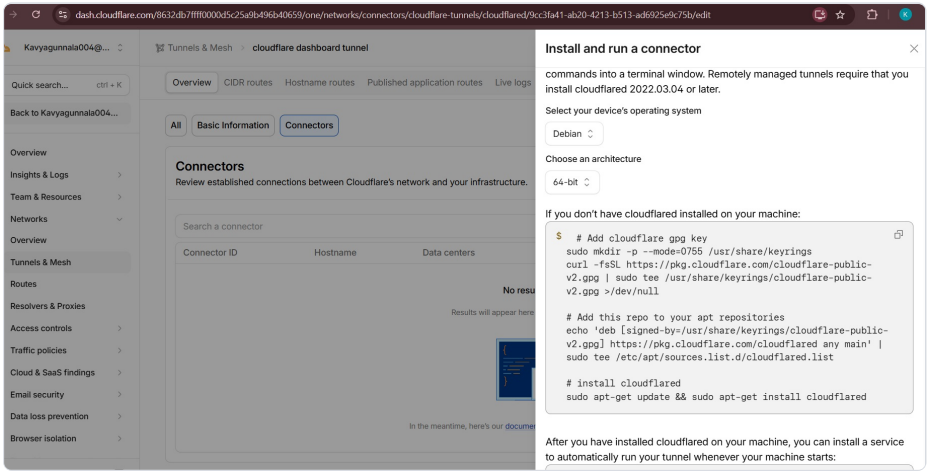




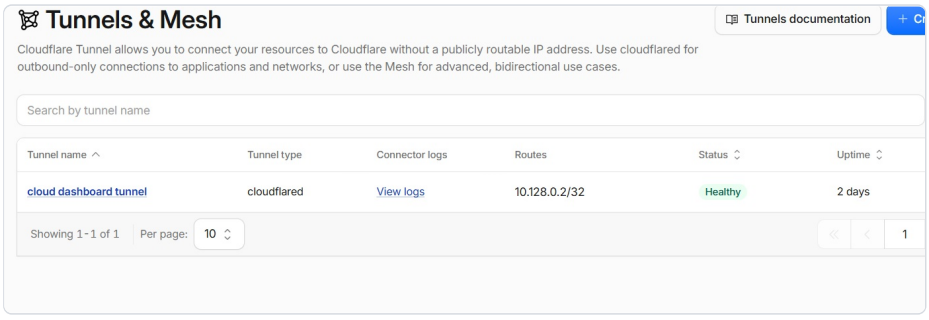




Tunnel was created but it is in Inactive status. Now we have to connect tunnel with the VM instance we created. Click on the tunnel name -> Connectors -> Add a connector. Select OS as Debian and the given commands in the SSH shell of the created VM instance.



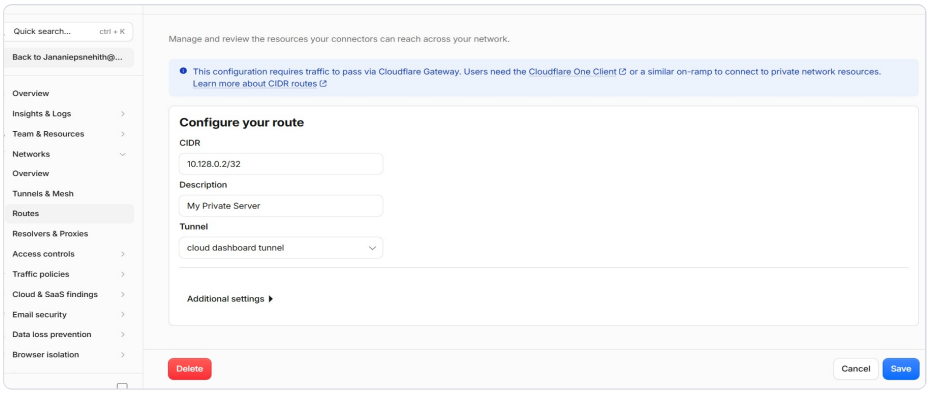
Then tunnel connects with the VM instance. You can verify by checking the status which shows as Healthy.



Step 3 - Configure the Private Network Route

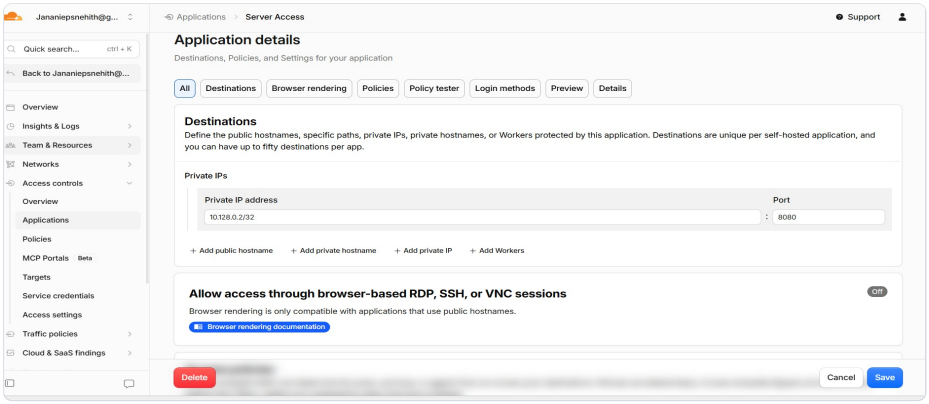
After establishing the tunnel, the next step was to configure a route for the private server.

Navigate to: **Networks → Routes → Add CIDR route**. A route tells Cloudflare: "Traffic intended for this private IP/subnet should be sent through the appropriate connector/tunnel."



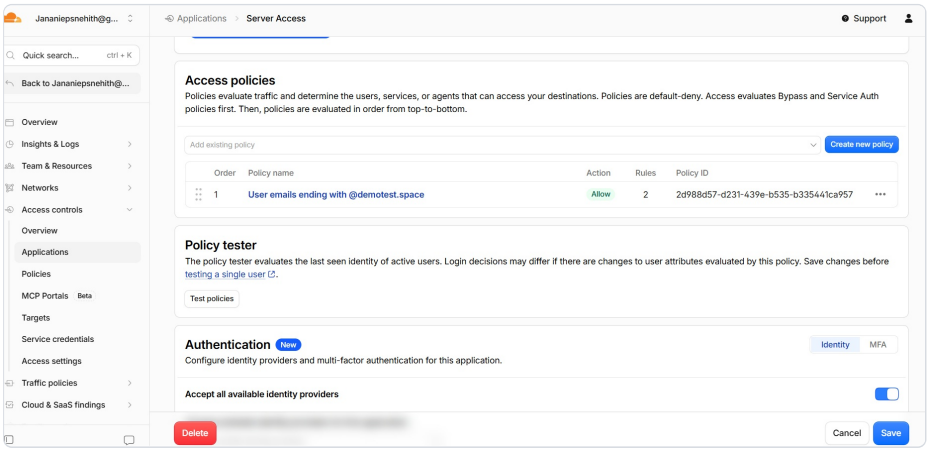
Step 4 - Create the Private Network Application

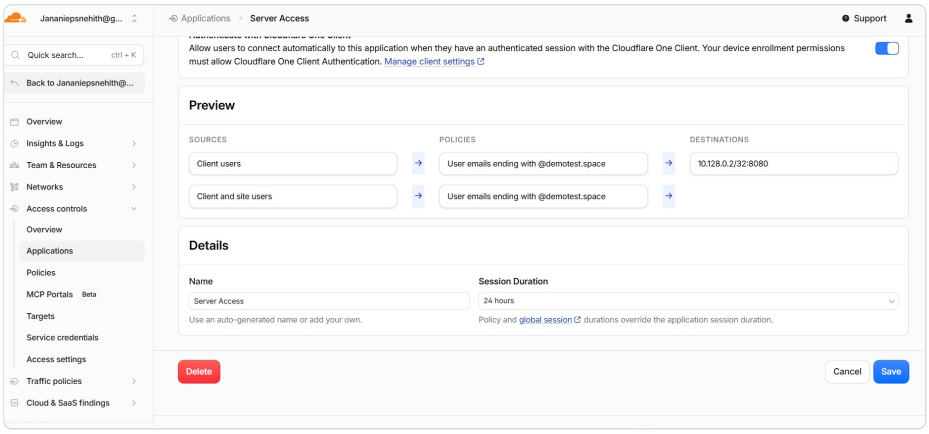
The next step was to define the protected resource in Cloudflare Zero Trust. Navigate to: Access → Applications. Create a new application and select: **Private Network**. This is important because the resource being protected is an **internal/private network resource**, rather than a publicly accessible web application. The application should point to the same private IP/network and port used by the test server.



Step 5 - Configure the Access Policy

After creating the Private Network application, an Access Policy was configured. The purpose of the policy is to specify **who is allowed to access the private resource**. The project requires the policy to allow the authorized test identity group coordinated with Team 1.

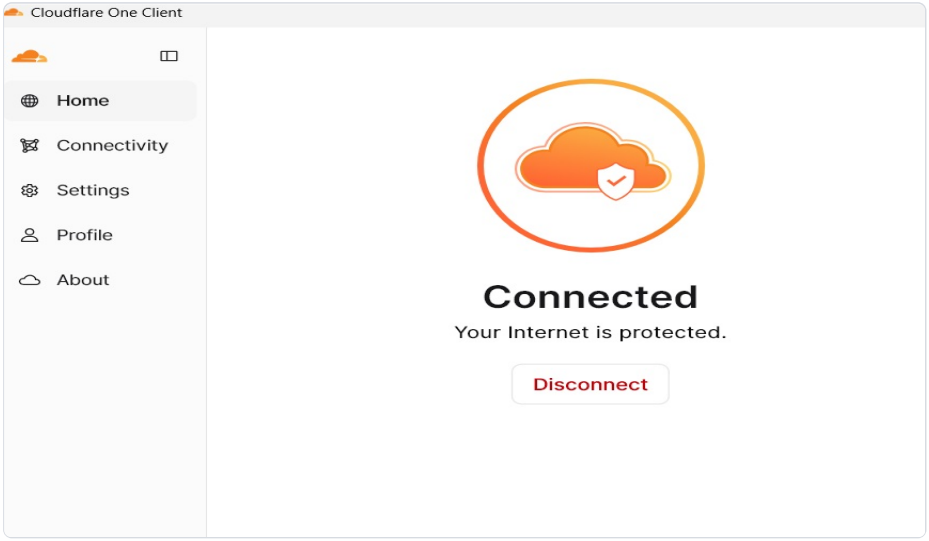


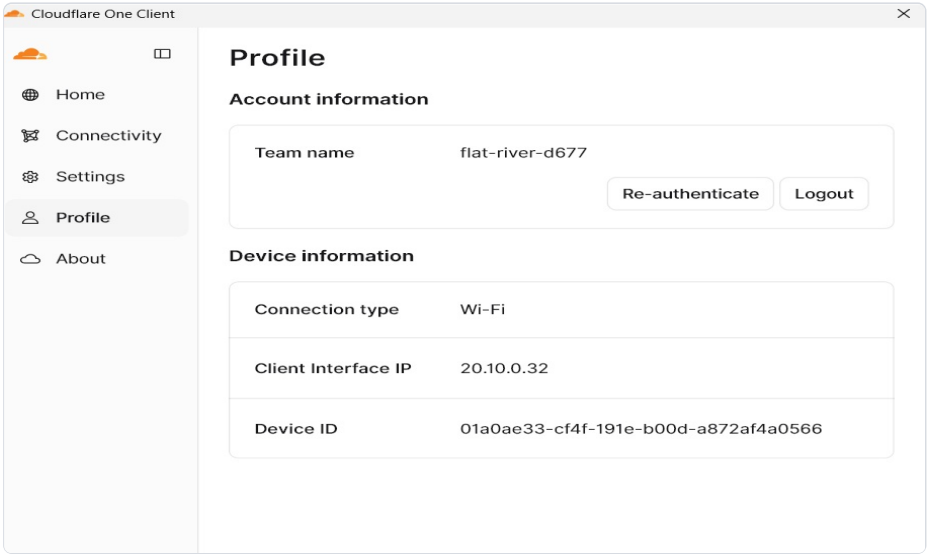


Step 6 - Enroll the Test Device

The next step was performed with Team 2. The test device needs to be enrolled and have WARP running. The overall connection becomes:

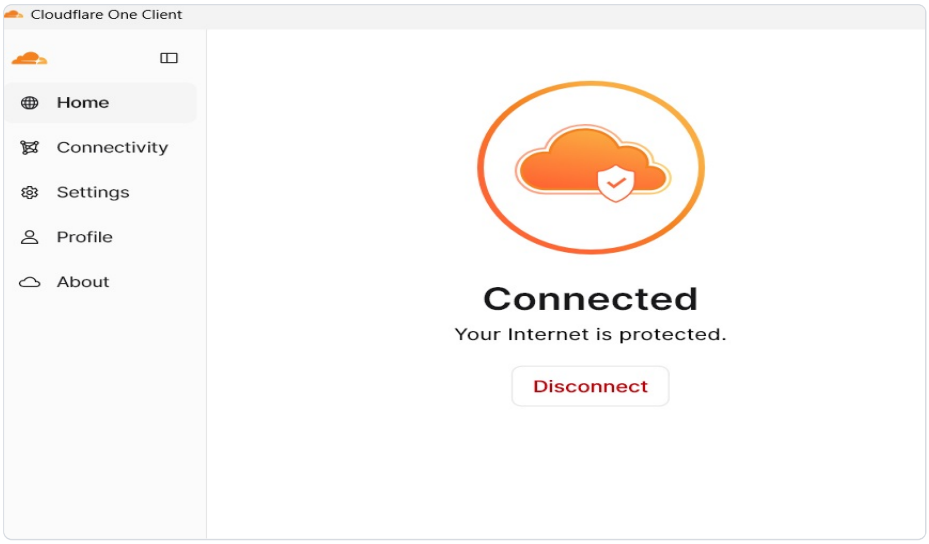
Test Device → WARP → Cloudflare → ZTNA Policy → Private Route → Tunnel → Private Server



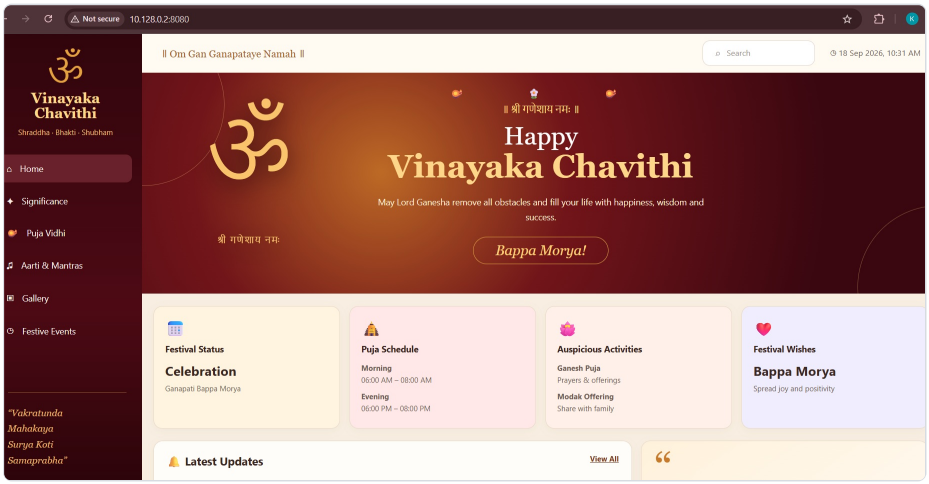


Step 7 - Test Access with WARP Enabled

With WARP enabled on the enrolled device, the user attempted to access the internal server using its private IP address.



Search the private IP in the web browser.



Expected result: The authorized enrolled device should be able to reach the private server.

WARP ON

↓

Cloudflare

↓

ZTNA Policy

↓

Private Server

Step 8 - Test Access with WARP Disabled

The same test was then repeated after disabling WARP. This is an important validation step because it demonstrates that the device cannot reach the protected private resource through the configured ZTNA path when WARP is disconnected**.**

WARP OFF

↓

Attempt to access private server

↓

No ZTNA/WARP path

↓

ACCESS BLOCKED / UNREACHABLE ✖

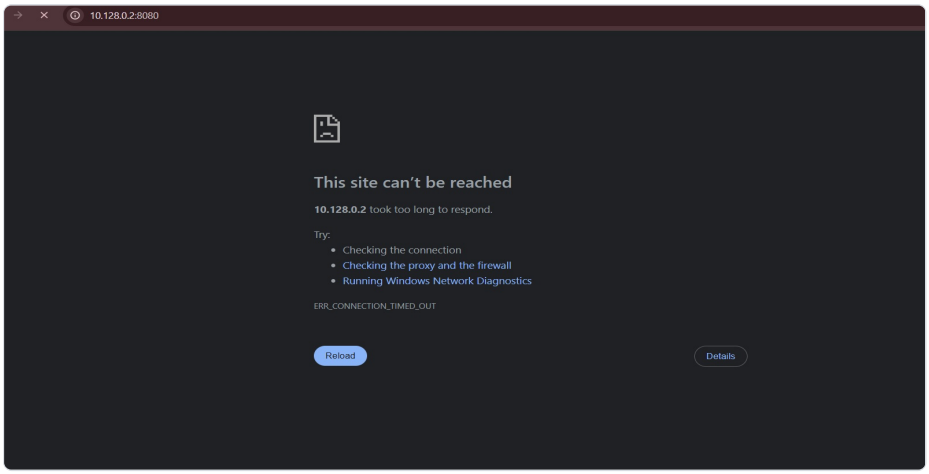


Figure: Private server unreachable after WARP was disabled.

Supplementary Deep-Dive: Sophos Firewall Integration with Microsoft Entra ID

Azure Deployment • Identity-Based Authentication • Remote-Access IPsec VPN

Implementation and Configuration Report

Project Area	Implemented Configuration
Firewall	Sophos Firewall deployed on Microsoft Azure
Identity	Microsoft Entra ID
Management	Sophos Central
VPN	Remote Access IPsec VPN
VPN Client	Sophos Connect
Cloud Platform	Microsoft Azure
Region	South India

Purpose: This document describes the implemented process for deploying Sophos Firewall on Azure, integrating Microsoft Entra ID for authentication and group-based access, configuring remote-access IPsec VPN, and validating secure access to the private Azure LAN.

1. Introduction

The project involved deploying Sophos Firewall on Microsoft Azure and integrating it with Microsoft Entra ID to provide secure remote-access IPsec VPN connectivity.

Major Components

- Sophos Firewall on Azure
- Azure Virtual Network and subnets
- Sophos Central management
- Microsoft Entra ID authentication
- Microsoft Entra ID group-based VPN access
- Remote Access IPsec VPN
- Sophos Connect client
- Azure Network Security Group (NSG) rules

- VPN-to-LAN firewall policy

High-Level Architecture

The implemented architecture provides a public entry point to the Sophos Firewall, a private Azure LAN, and a remote-access IPsec VPN for authorized users.

Component	Configuration
Azure Public IP	20.219.90.194
Sophos WAN	10.20.1.4
Azure VNet	10.20.0.0/16
Private LAN	10.20.2.0/24
VPN Client	Sophos Connect
Identity Provider	Microsoft Entra ID

Traffic flow: Internet → Azure Public IP → Sophos Firewall → IPsec VPN → private Azure LAN. Microsoft Entra ID supplies the organizational identity used for VPN authentication and group-based access.

2. Azure Infrastructure

A dedicated Azure environment was created for the Sophos Firewall. The Azure Virtual Network contains separate WAN and LAN subnets.

Component	Configuration
Resource Group	Cloud-sophos
Region	South India
VNet	vnet-cloud-sophos
Address Space	10.20.0.0/16
WAN Subnet	sophos-WAN
WAN Network	10.20.1.0/24
LAN Subnet	sophos-LAN
LAN Network	10.20.2.0/24

Sophos Firewall Azure Resources

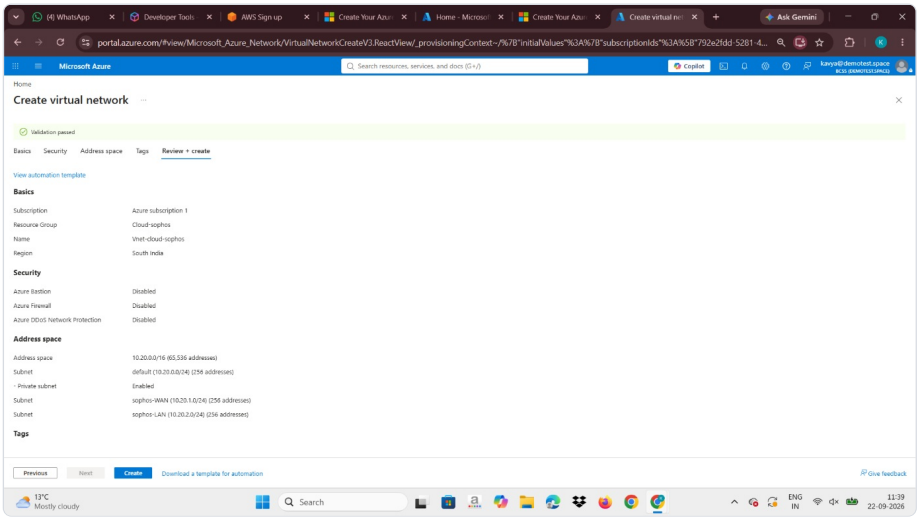
Resource	Configuration
Resource Group	rg-sophos-firewall
VM	sophos-sase-fw
Public IP	20.219.90.194
WAN Private IP	10.20.1.4

Step-by-Step

1. Create or use the dedicated Azure resource group for the Sophos environment.
2. Create the VNet vnet-cloud-sophos with address space 10.20.0.0/16.
3. Create the WAN subnet sophos-WAN using 10.20.1.0/24.

4. Create the LAN subnet sophos-LAN using 10.20.2.0/24.
5. Deploy the Sophos VM into the Azure environment.
6. Assign the public IP 20.219.90.194 and confirm the WAN private IP is 10.20.1.4.

[FIG : Azure VNet showing both WAN and LAN subnets]



[FIG: Sophos VM overview showing public/private IP]

3. Sophos Firewall Deployment

Sophos Firewall was deployed from the Azure Marketplace using the BYOL licensing model.

Deployment Details

Setting	Value
VM Name	sophos-sase-fw
Model	SFVUNL
SFOS Version	22.0.1 MR-1-Build490
VM Size	Standard_D2ls_v5
Region	South India
Licensing	BYOL

Storage

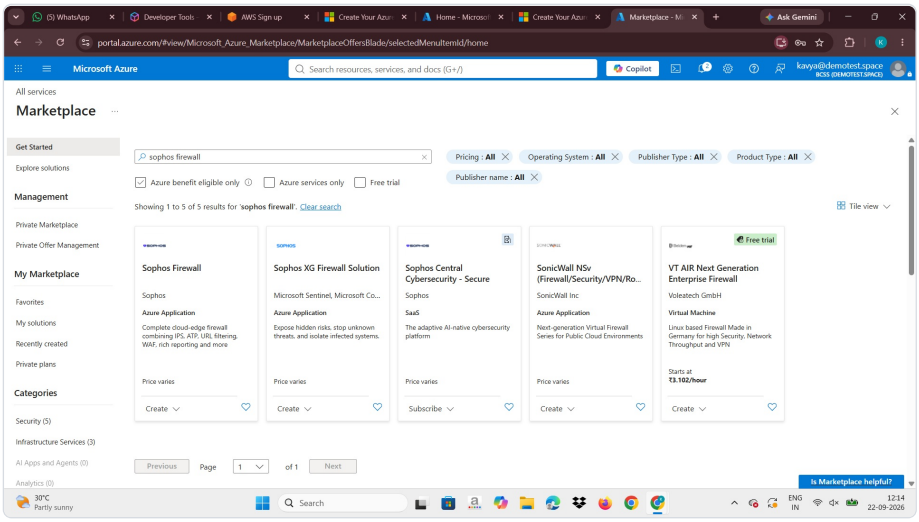
Setting	Value
Storage Account	sophosstorage2027
Performance	Standard
Redundancy	LRS

Step-by-Step Deployment

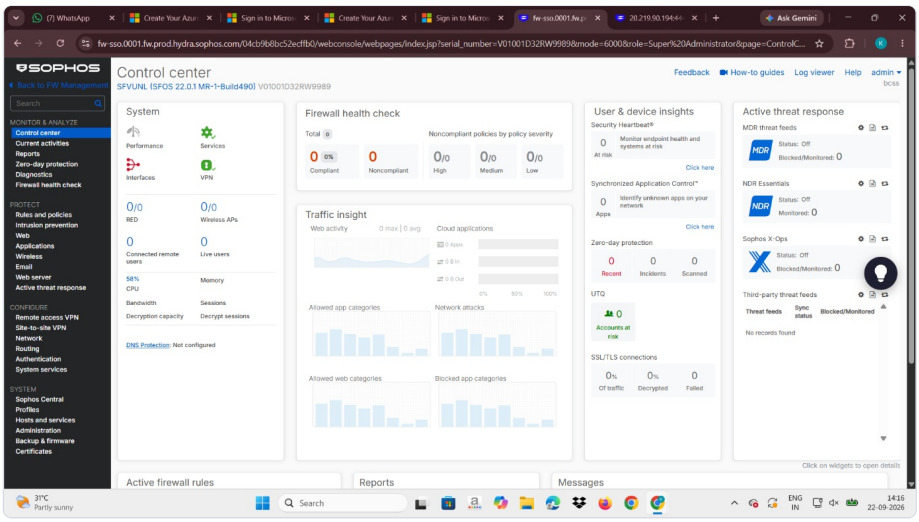
7. Open Microsoft Azure Marketplace.
8. Search for Sophos Firewall and select the Sophos Firewall virtual appliance.
9. Choose the BYOL licensing model.

0. Set the VM name to sophos-sase-fw.
1. Select the Standard_D2ls_v5 VM size.
2. Deploy the firewall in the South India region.
3. Configure the WAN and LAN network interfaces/subnets according to the Azure design.
4. Assign the public IP 20.219.90.194 to the WAN side.
5. After deployment, verify the WAN private IP 10.20.1.4.
6. Open the Sophos Web Admin Console using https://20.219.90.194:4444.
7. Complete the initial secure storage master key configuration.

[FIG: Azure Sophos Marketplace deployment]



[FIG: Sophos Firewall dashboard]



4. Sophos Central Management

The deployed Sophos Firewall was registered with Sophos Central to provide centralized management, monitoring, reporting, and configuration backup services.

Device Information

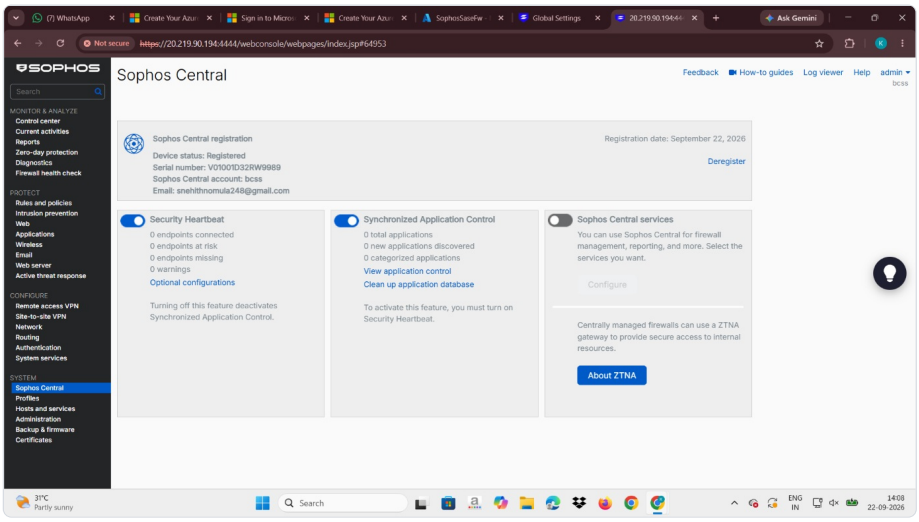
Field	Value
Serial	V01001D32RW9989
Model	SFVUNL
SFOS	22.0.1 MR-1-Build490
Management	Sophos Central

Step-by-Step

8. Sign in to Sophos Central.
9. Open the firewall management section.
0. Register/connect the deployed Sophos Firewall with Sophos Central.
1. Verify that the firewall appears as an online/connected device.
2. Confirm that centralized management and monitoring are available.
3. Enable or verify reporting and configuration backup services used by the project.

Successful registration confirms that the Azure-hosted Sophos Firewall can be centrally managed from Sophos Central.

[FIG: Sophos Central → Firewall Management showing the firewall as connected]



5. Microsoft Entra ID Authentication

Microsoft Entra ID was integrated with Sophos Firewall so that organizational identities could be used for authentication.

Step 1 — Create the Entra Application

4. Open Microsoft Entra ID in the Azure portal.
5. Go to App registrations.
6. Create an application for the Sophos Firewall integration.
7. Record the Client ID and Tenant ID.
8. Create a Client Secret and record it securely.

Step 2 — Configure API Permissions

The required Microsoft Graph permissions were configured for the application.

Permission	Purpose in the implementation
email	Email identity information
openid	OpenID authentication
profile	User profile information
User.Read	Read signed-in user information
User.ReadAll	Read user information required by the integration
Group.Read.All	Read groups for group-based access

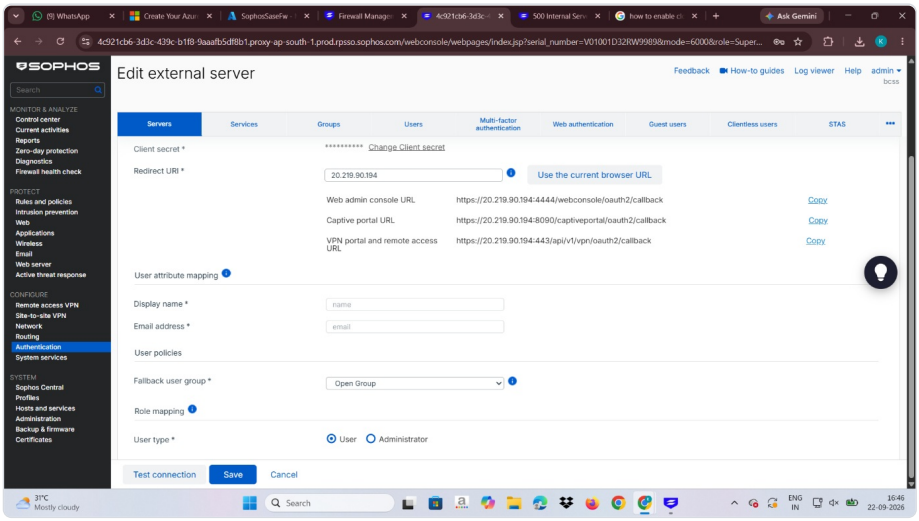
Administrator consent was granted for the configured permissions.

Step 3 — Configure Sophos

9. Open Sophos Firewall.
0. Go to Authentication → Servers.
1. Select Microsoft Entra ID.
2. Enter the Client ID, Tenant ID, and Client Secret from the Entra application.
3. Save the configuration.
4. Run the connection test.
5. Confirm that the connection test succeeds.

[FIG: Microsoft Entra API permissions showing administrator consent]

[FIG: Sophos Microsoft Entra server showing successful connection test]



6. Microsoft Entra Group Integration

A Microsoft Entra security group was imported into Sophos Firewall. The imported group was then used to control which users were permitted to access the remote-access VPN.

Step 1 — Import the Entra Group

6. Open the Microsoft Entra ID integration in Sophos Firewall.
7. Retrieve/import the required Microsoft Entra security group.
8. Verify that the group is visible in Sophos Firewall.
9. Use the imported group as the authorized group for VPN access.

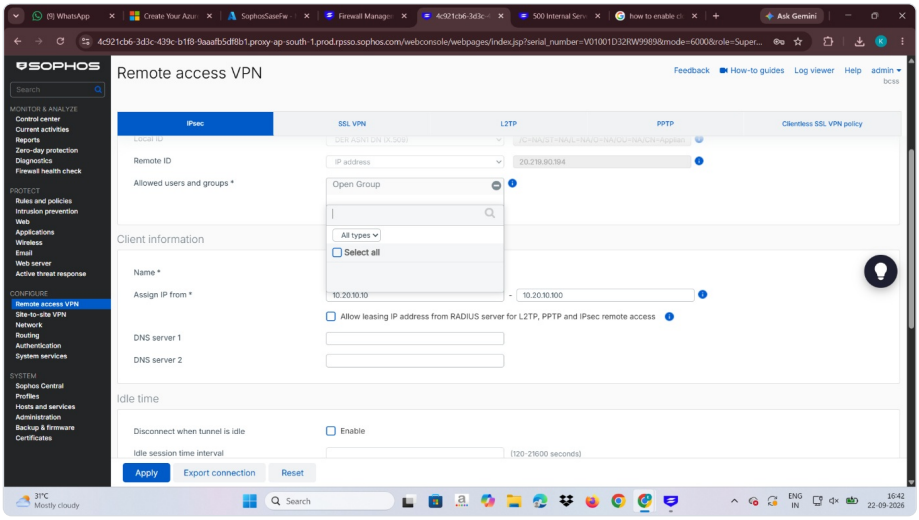
Step 2 — Configure Authentication Services

Service	Authentication
VPN Portal	Microsoft Entra ID
VPN/IPsec	Microsoft Entra ID
Firewall Administration	Local
Administrator Authentication	Local

This configuration makes Microsoft Entra ID the authentication source for VPN-related access while local authentication remains configured for firewall administration.

[FIG: Imported Microsoft Entra group]

[FIG: Authentication → Services showing Microsoft Entra ID selected for VPN]



7. Remote Access IPsec VPN

Remote Access IPsec VPN was enabled on Sophos Firewall to provide secure remote connectivity to the private Azure LAN.

VPN Configuration

Setting	Value
Interface	PortB - 10.20.1.4
IPsec Profile	DefaultRemoteAccess
Authentication	Digital Certificate
VPN Client Pool	10.20.10.10 – 10.20.10.100
Permitted Network	10.20.2.0/24
VPN Certificate	sophos-remote-vpn
Allowed users/groups	Imported Microsoft Entra group

Step-by-Step

0. Open Sophos Firewall and navigate to the Remote Access VPN/IPsec configuration.
1. Enable Remote Access IPsec VPN.
2. Select PortB with WAN private IP 10.20.1.4 as the configured interface.

3. Use the DefaultRemoteAccess IPsec profile.
4. Configure Digital Certificate authentication.
5. Create/configure the local certificate named sophos-remote-vpn.
6. Set the VPN client address pool to 10.20.10.10–10.20.10.100.
7. Set the permitted private network to 10.20.2.0/24.
8. Select the imported Microsoft Entra group under allowed users/groups.
9. Enable the required IPsec access from the WAN interface under Administration → Device Access.
0. Save the configuration.

[Screenshot : IPsec Remote Access configuration]

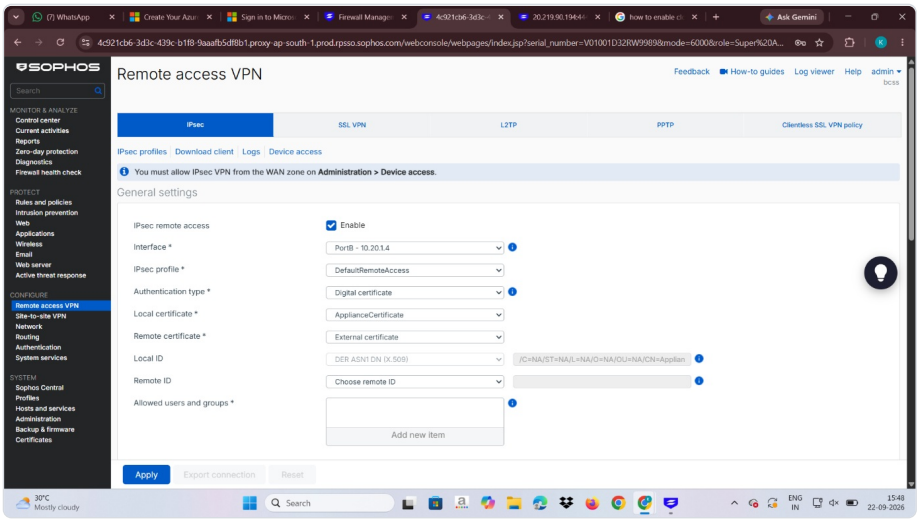
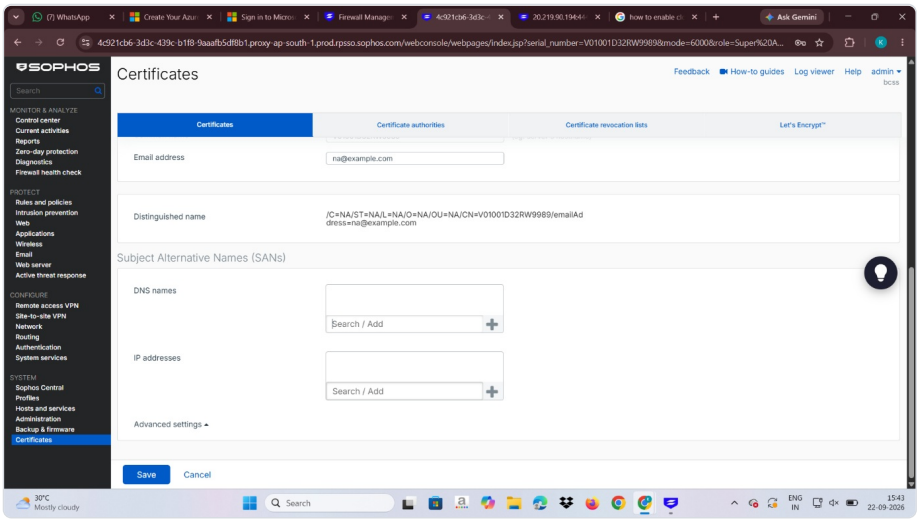


FIG: VPN certificate and Device Access configuration



8. Network Security Configuration

Azure Network Security Group rules were configured to permit the required VPN-related traffic to reach the Sophos Firewall.

Azure NSG Rules

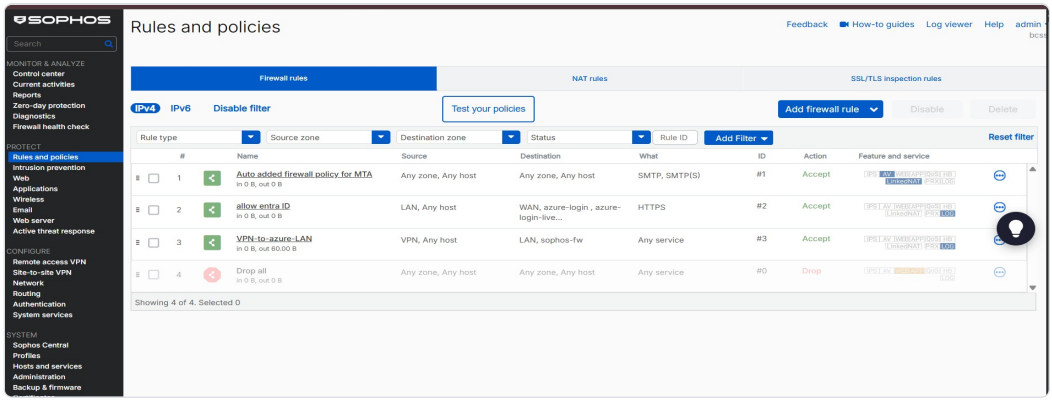
Protocol	Port	Purpose
TCP	443	VPN Portal / HTTPS
UDP	500	IKE
UDP	4500	IPsec NAT-T

Sophos VPN-to-LAN Firewall Rule

Setting	Configuration
Rule Name	VPN-to-azure-LAN
Source Zone	VPN
Destination Zone	LAN
Destination	sophos-fw
Service	Any
Action	Accept
Logging	Enabled
Rule Position	Above final Drop all rule

Step-by-Step

1. Open the Azure Network Security Group associated with the Sophos deployment.
2. Add/verify inbound TCP 443 for VPN Portal/HTTPS.
3. Add/verify inbound UDP 500 for IKE.
4. Add/verify inbound UDP 4500 for IPsec NAT-T.
5. In Sophos Firewall, create the VPN-to-azure-LAN firewall rule.
6. Set the source zone to VPN and destination zone to LAN.
7. Set the destination to sophos-fw as configured in the implementation.
8. Set the service to Any as used in the implementation.
9. Set the action to Accept and enable logging.
0. Place the rule above the final Drop all rule.



9. Sophos Connect Client

The Sophos Connect client was installed on Windows to establish the remote-access IPsec VPN.

Step-by-Step

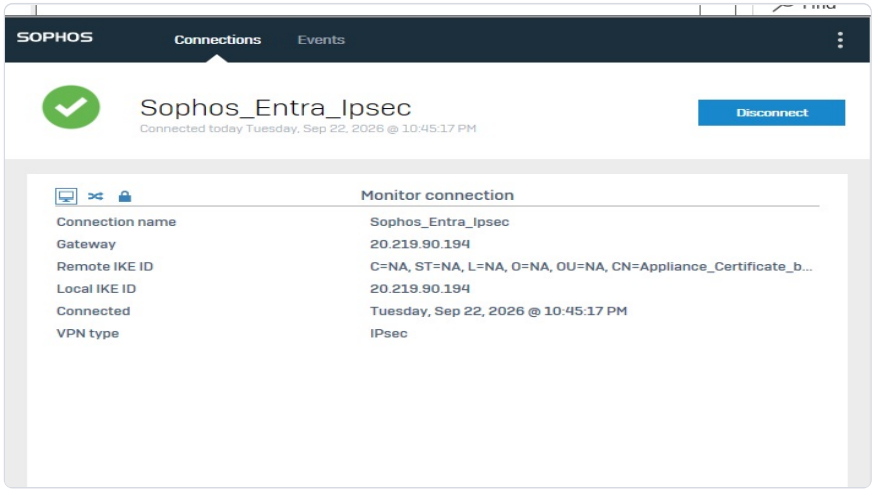
1. Install the Sophos Connect client on the Windows device.
2. Import the IPsec configuration generated for the Sophos Firewall.
3. Configure the VPN gateway to use the public IP 20.219.90.194.
4. Start the VPN connection.
5. Authenticate using the configured Microsoft Entra identity/group.
6. Verify that the VPN connection is established.
7. Confirm that the client receives an address from the VPN pool.
8. Verify that the client receives a route to the private LAN 10.20.2.0/24.

Observed Result

Item	Observed Value
VPN Gateway	20.219.90.194
Assigned VPN IP	10.20.10.10
Private route	10.20.2.0/24
VPN Client	Sophos Connect

A second Windows client was also successfully connected, demonstrating that more than one authorized client could establish the configured remote-access VPN.

[FIG: Sophos Connect showing connected status]



10. Testing Results

Test / Component	Result
Sophos Firewall deployment	✓ Successful
Sophos Central registration	✓ Successful
Microsoft Entra ID integration	✓ Successful
Entra administrator consent	✓ Successful
Entra group import	✓ Successful
VPN Portal access	✓ Successful
IPsec VPN configuration	✓ Successful
Sophos Connect connection	✓ Successful
VPN IP assignment	✓ Successful
VPN route	✓ Successful
Second VPN client	✓ Successful

Final Result

The project successfully implemented a Sophos Firewall-based remote-access VPN solution in Azure. Microsoft Entra ID was integrated for identity-based authentication, while Sophos Connect was used to establish an IPsec VPN connection. Authorized users received VPN addresses from the configured pool and were provided access to the private Azure network 10.20.2.0/24.

Implementation Demonstrates

- Cloud firewall deployment
- Network segmentation
- Identity-based authentication
- Certificate-based VPN
- Secure remote access
- Firewall policy enforcement
- Centralized Sophos management

Future Enhancement

The next stage can integrate the Sophos Firewall with Cloudflare SASE/WAN when the required Cloudflare WAN/IPsec functionality is available in the organization’s Cloudflare plan.

Team 4 — APPLICATION

Step-by-Step Implementation & Verification Procedure

Zero Trust / SASE Capstone — Cloudflare

Team	Team 04 — Application
Students	Benduri Akshaya, Duvvapu Jahnav Anand Patel
Application / Public Hostname	dash.demotest.space
Internal Origin	http://localhost:8080

1. Objective

The objective is to place an authentication-protected front door in front of an internal web application. The application is made reachable through a public URL while the real internal server address is not exposed directly. Only identities allowed by the Cloudflare Access policy should reach the application.

2. Implementation Flow

- Verify that the internal web application is running.
- Open Cloudflare Zero Trust and create a Self-hosted application.
- Configure the application hostname and destination.
- Publish the application route through the Cloudflare Tunnel.
- Create an Access Policy that allows only the approved identity/domain.
- Use Policy Tester to verify allowed and blocked users.
- Open the public URL and verify the Cloudflare Access login and successful authenticated access.

3. Step-by-Step Procedure

Step 1 — Verify the Internal Web Application

Before publishing the application, confirm that the internal web application is running and can be opened locally.

1. Open the application on the machine where the internal service is running.
2. Verify that the application page loads successfully.
3. In this implementation, the application is served from the local origin http://localhost:8080.

Expected result: The internal application page loads successfully before it is exposed through the Cloudflare-protected public URL.

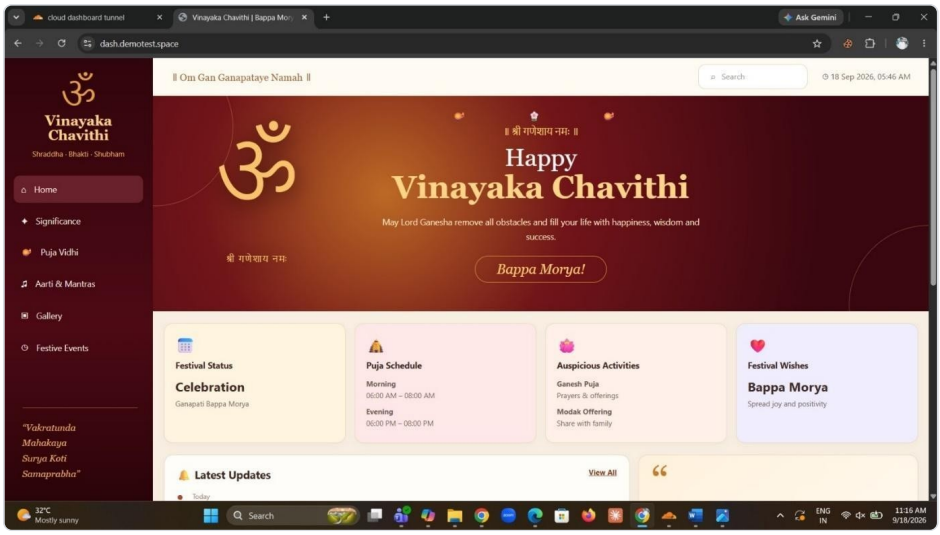
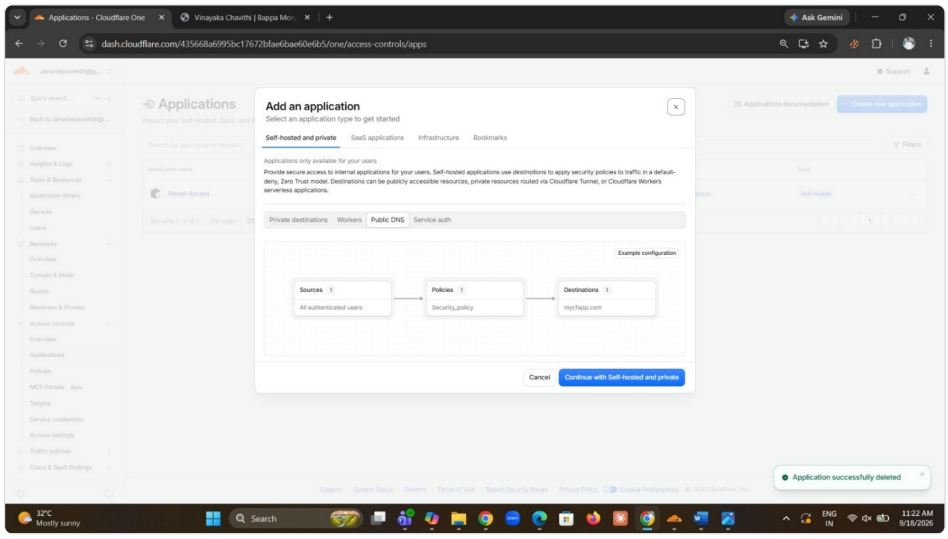


Figure 1. Internal web ap plication running successfuly.

Step 2 — Open Applications and Start a Self-hosted Application

4. Sign in to the Cloudflare Zero Trust dashboard.
5. Open Access controls → Applications.
6. Click Add an application.
7. Select Self-hosted.

Expected result: The Self-hosted application configuration screen opens.



Step 3 — Configure the Application

Configure the application so Cloudflare can identify the public destination and apply access control to it.

8. Enter the application details in the Self-hosted application wizard.
9. Configure the public hostname/subdomain used to access the application.
0. Use the application hostname dash.demotest.space.
1. Configure the destination/origin so the request is forwarded to the internal application.
2. Continue through the wizard and save the application configuration.

Expected result: Cloudflare creates the self-hosted application and shows it in the Applications section.

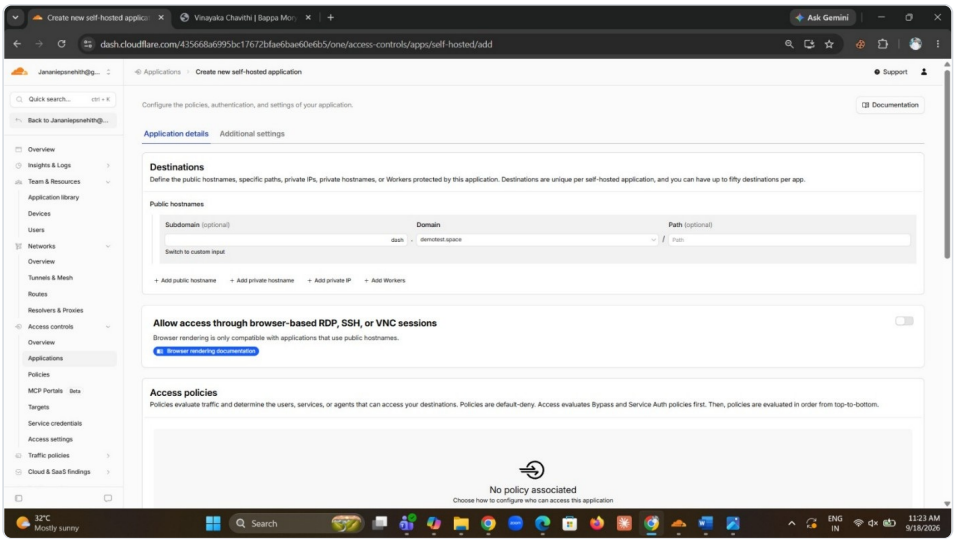


Figure 3. Self-hosted ap plication configuration screen.

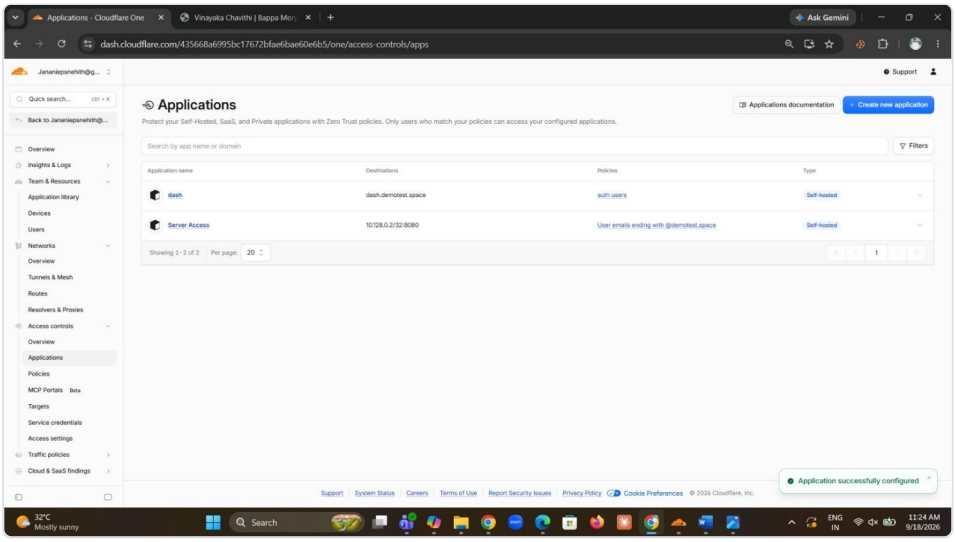


Figure 4. Ap plication successfuly configured and listed under the application library.

Step 4 — Publish the Application Route Through the Tunnel

The application need to be connected to the internal service, including the case where the service is behind Team 03's tunnel.

3. Open Networks → Tunnels & Mesh / published application routes.
4. Select the Cloudflare Dashboard Tunnel used for the application.
5. Choose Add a published application route.
6. Enter the public hostname dash.demotest.space.
7. Leave the path empty when the complete hostname should match the application.
8. Set the service/origin to the internal application address. In the completed setup, the route points to http://localhost:8080.
9. Save the route.

Expected result: The published route appears in the tunnel configuration and connects the public hostname to the internal application.

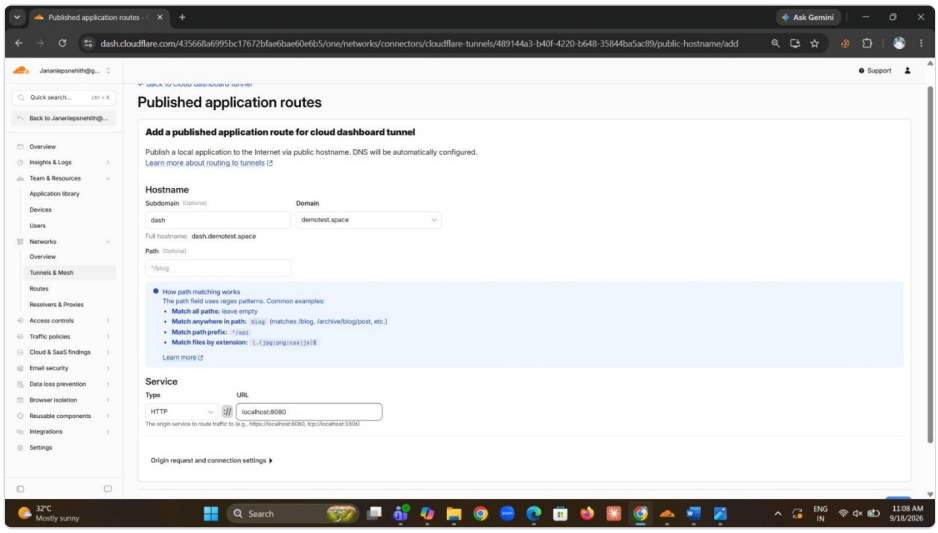


Figure 5. Published ap plication route configuration screen.

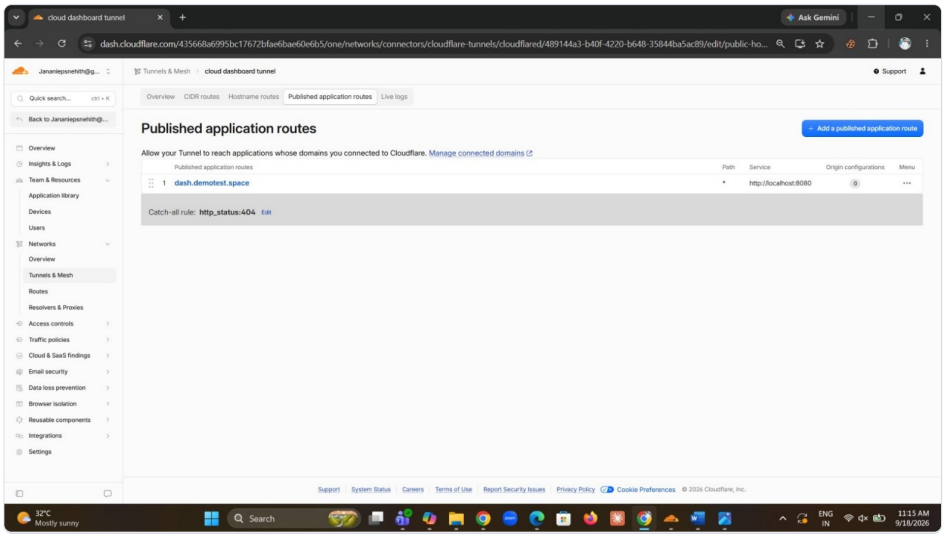


Figure 6. Published route showing dash.demotest.space map pedto http://localhost:8080.

Step 5 — Create the Cloudflare Access Policy

The application must not be openly accessible to everyone so create a policy and “Allow” policy restricted to the approved test identity group.

0. Open Access controls → Policies.
1. Create or edit the policy associated with the application.
2. Set the Action to Allow.
3. Under Include, select the required identity condition. In the completed setup, the condition is Emails ending in @demotest.space.
4. Save the policy.

Expected result: Only users whose identity matches the configured allow condition can pass the access policy.

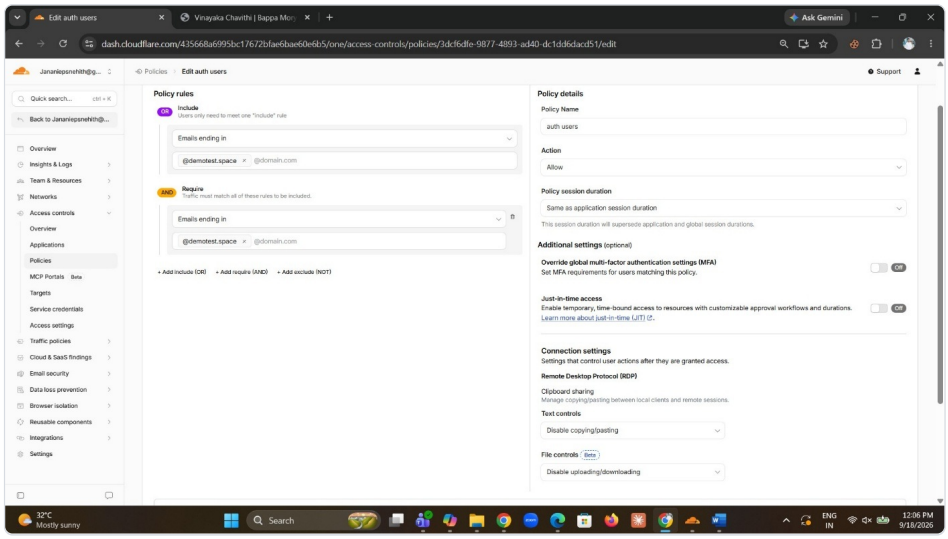


Figure 7. Access policy configured with an Allow action and an email-based Include rule.

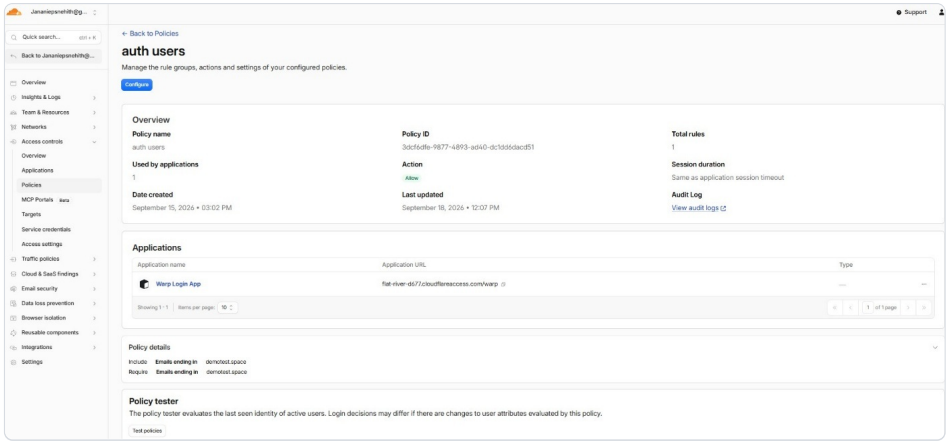


Figure 9. Saved policy detailsshowingthe configuredidentityrule and Alow action.



Figure 1 0. Ac ces s Policieslist showingthe active auth users policy.

Step 6 — Test the Access Policy

Use the built-in Policy Tester to confirm that the policy behaves differently for allowed and non-allowed identities.

5. Open the auth users policy.
6. Run the Policy Tester.
7. Review the approved users section and the blocked users section.
8. Confirm that users with the approved **@demotest.space** identity pattern are allowed.
9. Confirm that users outside the approved identity pattern are blocked.

Expected result: The policy tester shows both allowed and blocked results, demonstrating that the access rule is being enforced.

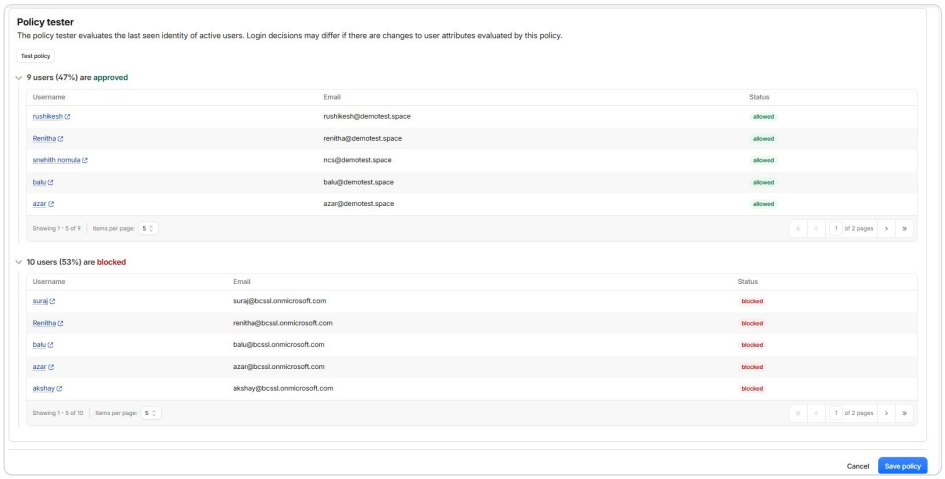


Figure 8. Policy Tester showing ap proved and blocked users.

Step 7 — Verify Authentication From the Public URL

Finally, test the application from the public hostname. The guide requires an unauthenticated user to see the Cloudflare Access login page before reaching the application.

0. Open a normal browser window and visit the public URL for the application.
1. Confirm that Cloudflare Access displays a login page instead of opening the application directly.
2. Select the configured identity sign-in method.
3. Sign in using an allowed account, such as the configured @demotest.space account.
4. Complete the identity-provider sign-in steps.
5. After successful authentication, verify that the actual internal web application opens.

Expected result: The user is authenticated first and is then allowed to reach the application. A user who does not satisfy the policy should not reach the application.

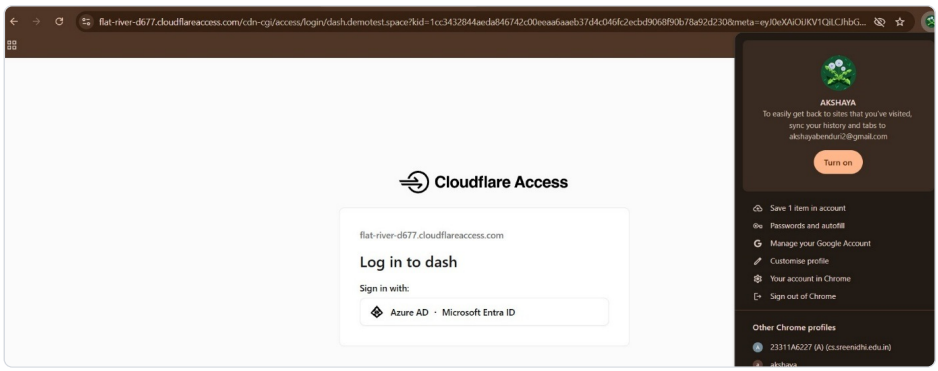


Figure11. Cloudflare Access login page displayed before ap plication access.

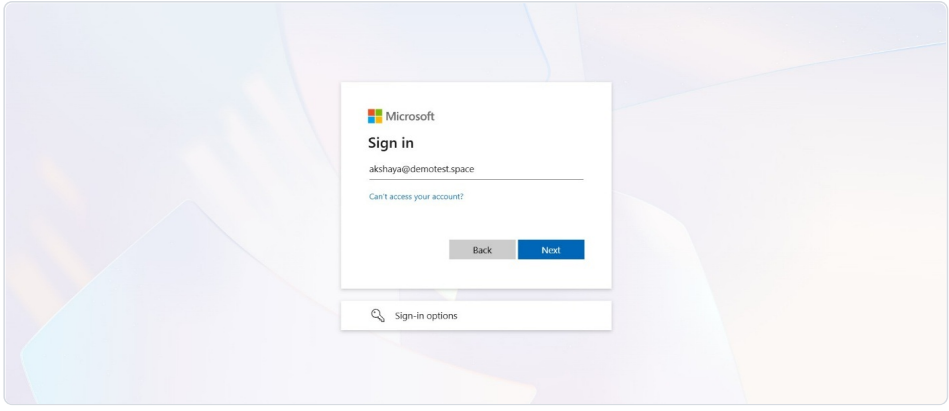


Figure 12. Identity-provider sign-in page.

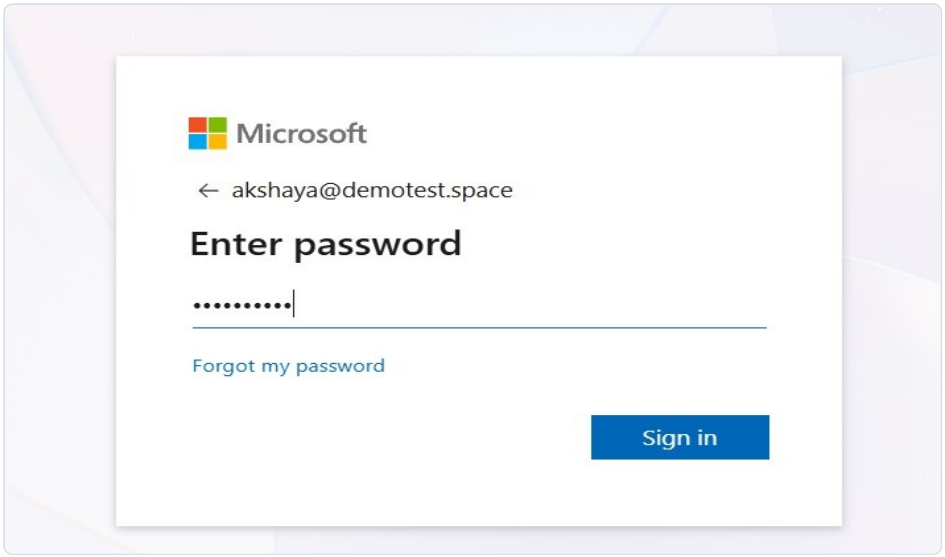


Figure 1 3. Pas sword verification step.



Figure 1 4. MFA approval step.

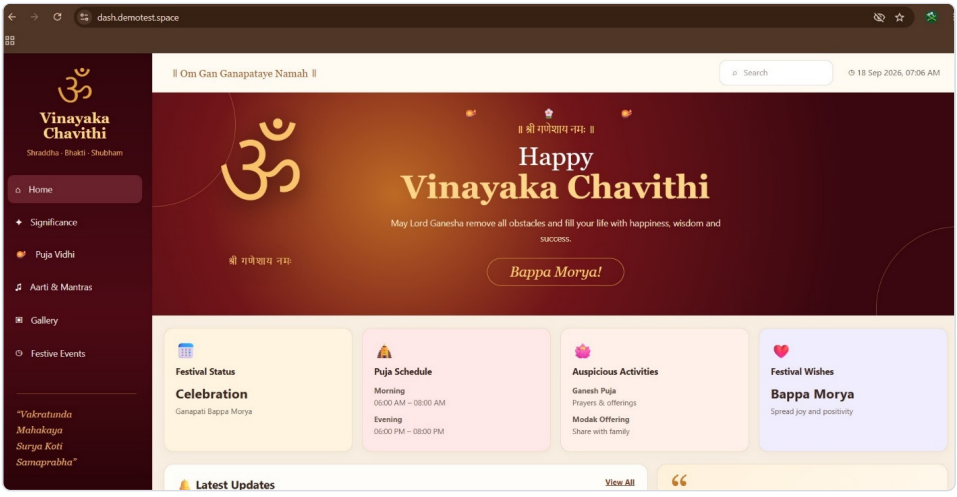


Figure 1 5. Ap plication successfuly opened after authentication.

4. Final Verification / Completion Criteria

- A public application URL is available for the internal application.
- Opening the public URL first displays the Cloudflare Access authentication page.
- Only identities matching the configured Access Policy can continue to the application.
- Policy Tester shows the expected allowed and blocked users.
- The internal service is reached through the configured tunnel/route rather than exposing its private address directly to the browser.
- Successful authentication, including MFA where configured, results in access to the actual application.

5. Result

Successfully configured a Self-hosted application protected by Cloudflare Access. The application is published through the public hostname dash.demotest.space, routed to the internal service, and protected by an identity-based Allow policy. Policy testing and end-to-end browser testing confirm that authentication is required before the application can be accessed.

Team 5 — WEB / INTERNET (SWG)

Cloudflare Zero Trust
Secure Web Gateway Implementation

Step-by-Step Procedure and Progress Report

Project	**Building a Zero Trust / SASE Architecture on Cloudflare's Free Tier**
Team Task	Team 5 — Web/Internet policies (SWG)
Team	V Rushikesh, G Naveen Reddy
Scope	DNS filtering, HTTP filtering, executable-download blocking, and security validation

1. Introduction

As a SOC Interns, we working on Web/Internet (SWG) layer, configured and validated web-traffic security controls in Cloudflare Zero Trust. Documented my work in the same order in which I performed the configuration and testing.

2. Objective

Main objective was to configure SWG controls for DNS and HTTP traffic, apply security filtering conditions, block selected malicious or risky traffic, and verify the results through Cloudflare policy dashboards, query/request logs, and browser-side blocking.

3. Evidence Selection

This report contains only screenshots that show our actual configuration inputs, policy progress, or enforcement outputs.

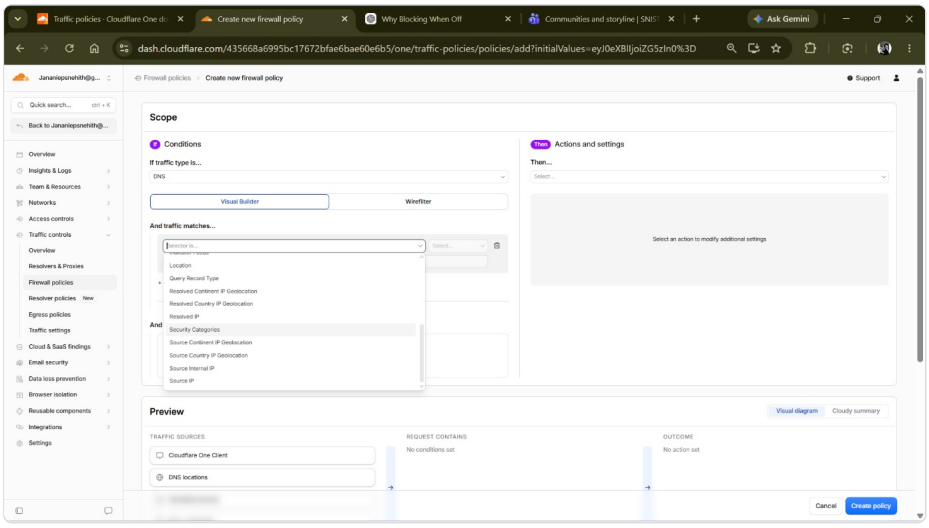
4. Step-by-Step Implementation and Progress

Step 1: Initial DNS Policy Status

First we checked the DNS Policies section to understand the current policy state. This gave the baseline before creating and validating the SWG controls for the project.

Step 2: Create the DNS Security Policy

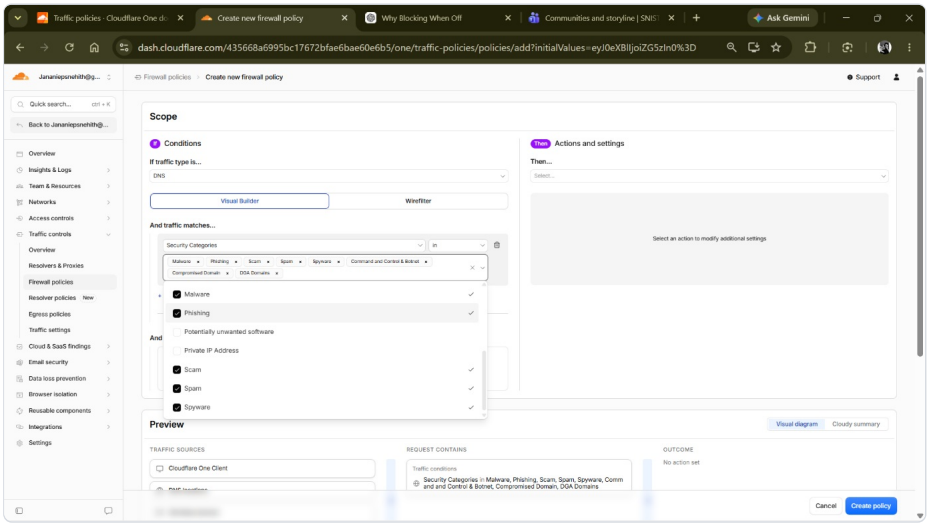
We started creating a DNS firewall policy. selected DNS traffic and defined the scope and conditions required for security filtering. The main goal was to inspect DNS requests and apply a blocking action when the selected security conditions were matched.



DNS firewall policy creation screen with the policy scope, DNS traffic type, and rule conditions.

Step 3: Select Security Categories

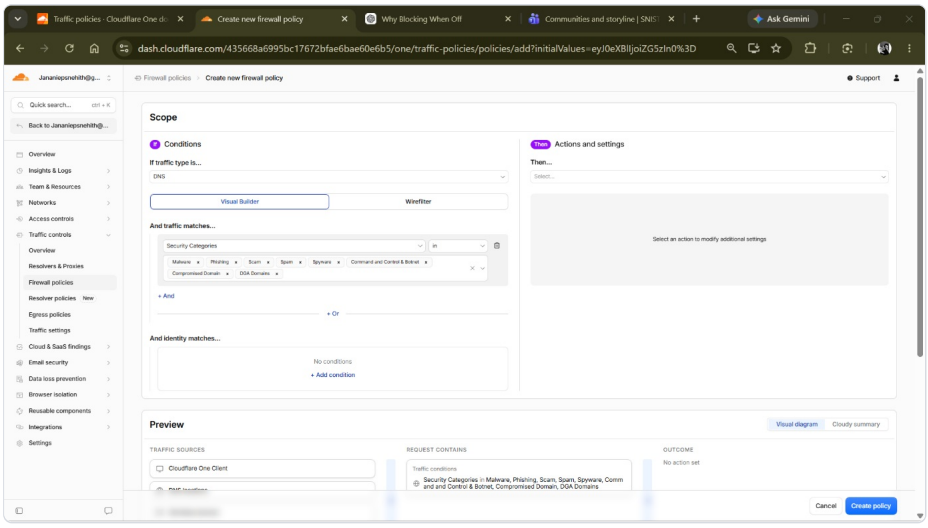
For the rule conditions, selected security-related categories available in the dashboard, including Malware, Phishing, Scam, Spam, Spyware, Command and Control & Botnet, Compromised Domain, and DGA Domains. These selections formed the main input for the malicious-domain blocking policy.



Security-category inputs selected for the DNS filtering rule.

Step 4: Confirm Security Category Configuration

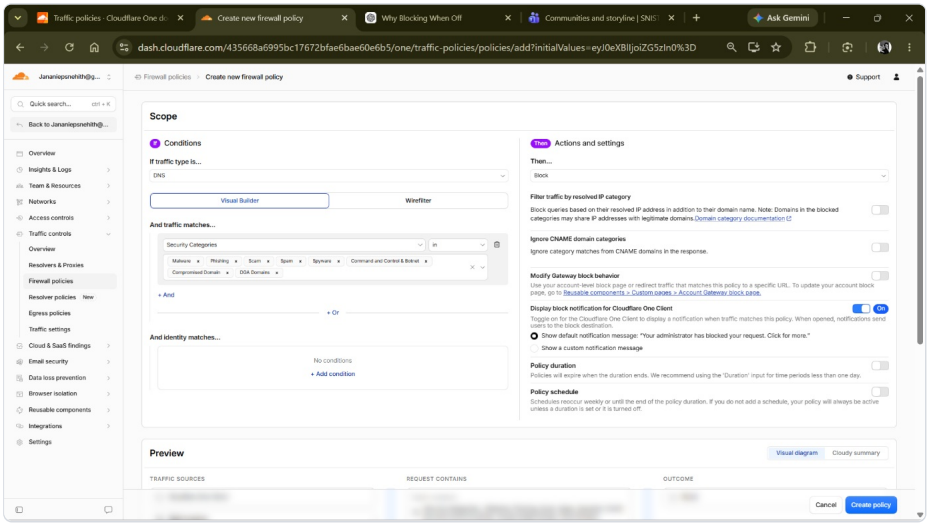
Then reviewed the selected security categories and verified that the intended categories were part of the rule before proceeding. This was my configuration checkpoint before finalizing the policy.



DNS security-category configuration continued before saving the policy.

Step 5: Configure the DNS Policy Action

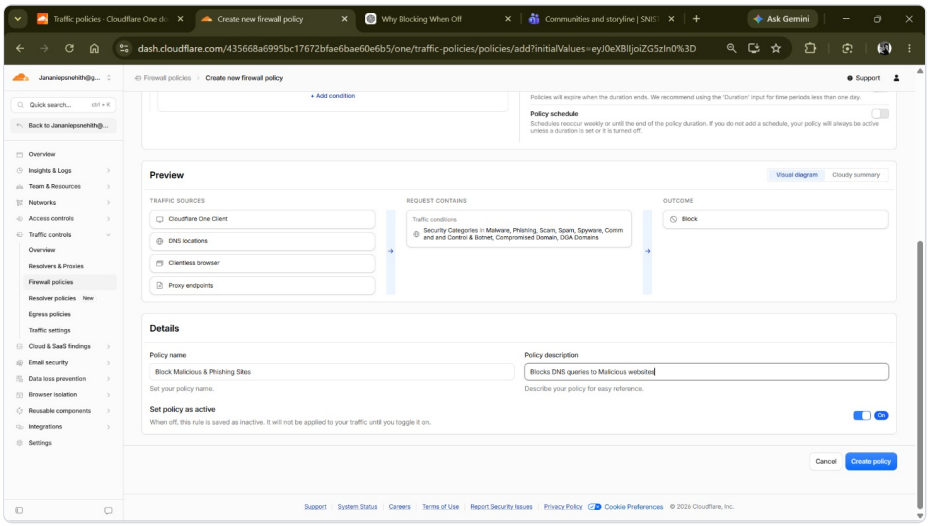
we completed the DNS policy configuration and set the intended enforcement action to Block. At this stage, we were converting the selected security categories into an active SWG control.



DNS policy configuration with the selected matching conditions and blocking action.

Step 6: Review DNS Policy Before Applying

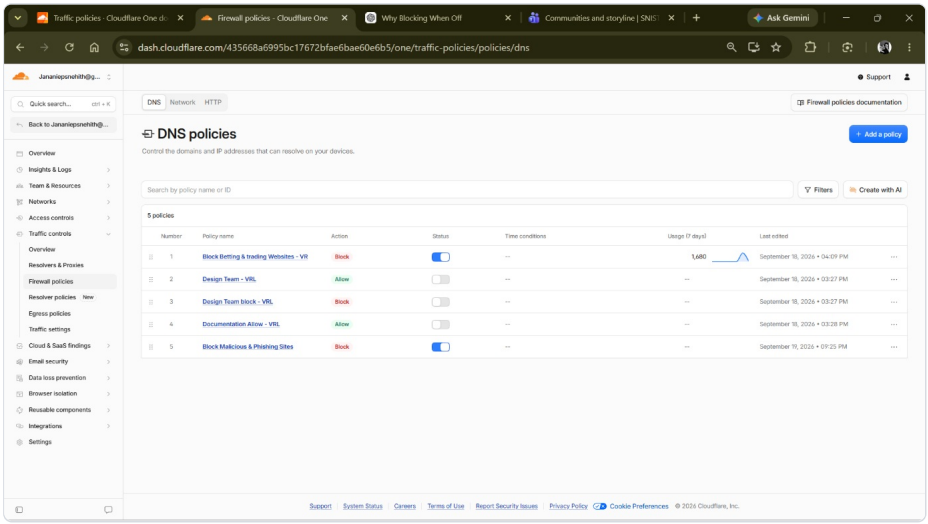
Before applying the rule, reviewed the policy preview. The preview confirmed the traffic source, the security-category conditions, and the Block outcome. This helped us to verify that the rule matched the configuration that intended to deploy.



DNS policy preview showing the traffic conditions, selected security categories, and Block outcome.

Step 7: DNS Policy Added

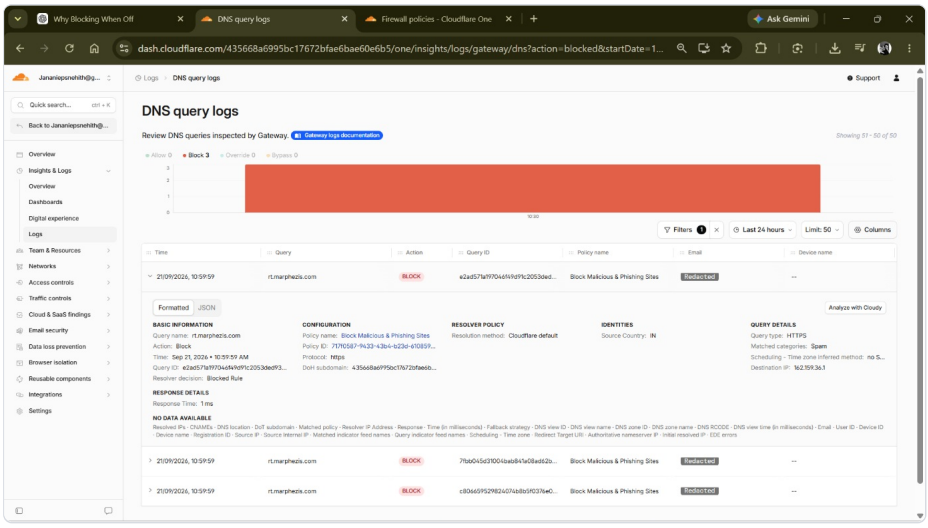
After saving the configuration, returned to the DNS Policies dashboard and verified that the policy was present. This was progress checkpoint showing that the DNS filtering control had been added.



DNS Policies dashboard after the security filtering policy was configured.

Step 8: DNS Blocking Test — Spam Category

We tried to get access to test malicious sources, then checked the DNS query logs to validate enforcement. A request to **rt.marphezis.com** was recorded with the action Block. The log identified the applied policy as **Block Malicious & Phishing Sites** and the **category as Spam**. This provided evidence that the DNS filtering rule was actively blocking matching traffic.

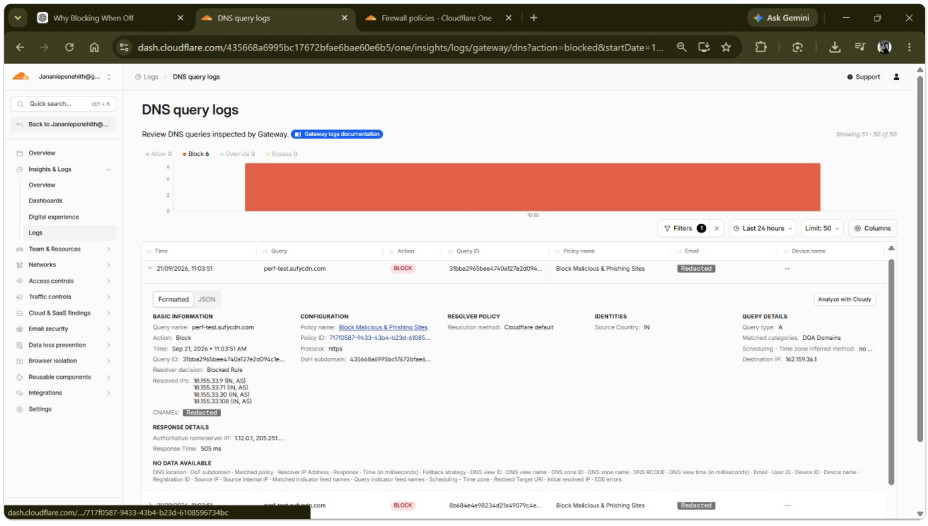


DNS query log showing a request blocked by the malicious/phishing policy and categorized as Spam.

Result: DNS request was blocked and categorized as Spam.

Step 9: DNS Blocking Test — DGA Domain

Also performed another DNS validation using the query log. The request to **perf-test.sufycdn.com** was blocked under the **Block Malicious & Phishing Sites policy**, with the resolver decision shown as Blocked Rule and the category identified as **DGA Domains**. This gave me a second enforcement result for the DNS layer.

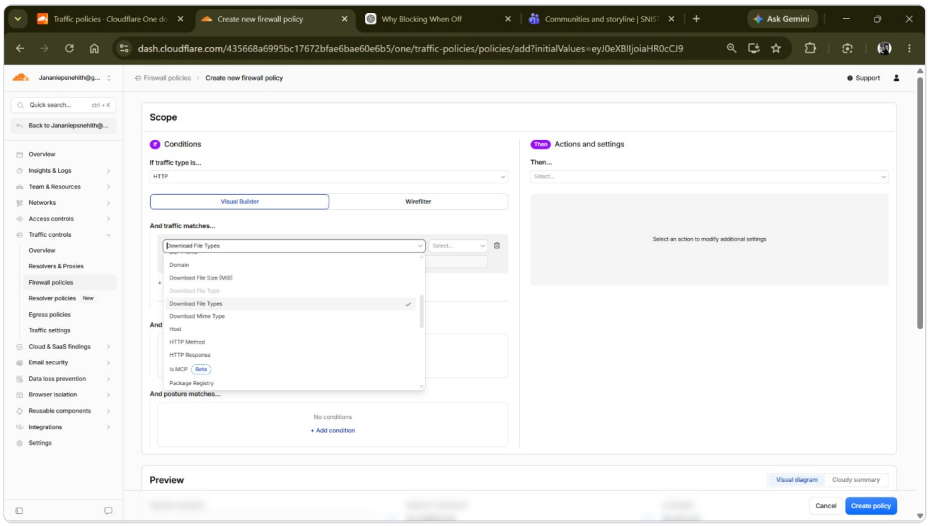


DNS query log showing a request blocked under the DGA Domains category.

Result: DNS request was blocked and categorized as DGA Domains.

Step 10: Create the HTTP Filtering Policy

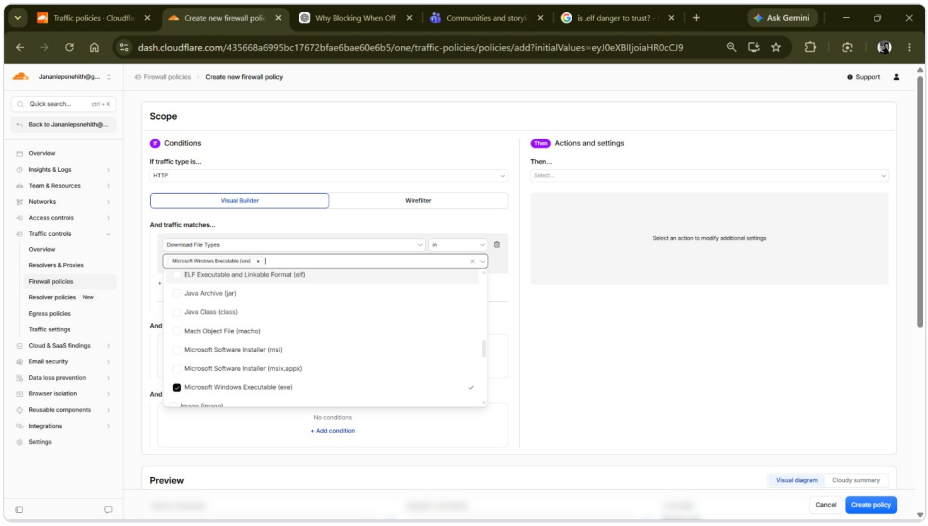
After validating DNS filtering, moved to **HTTP filtering**. then created an **HTTP policy** and selected **Download File Types** as the condition because I wanted to control **executable file downloads** at the web-traffic layer.



HTTP policy builder with HTTP traffic selected and Download File Types used as the filtering condition.

Step 11: Select Windows Executable File Type

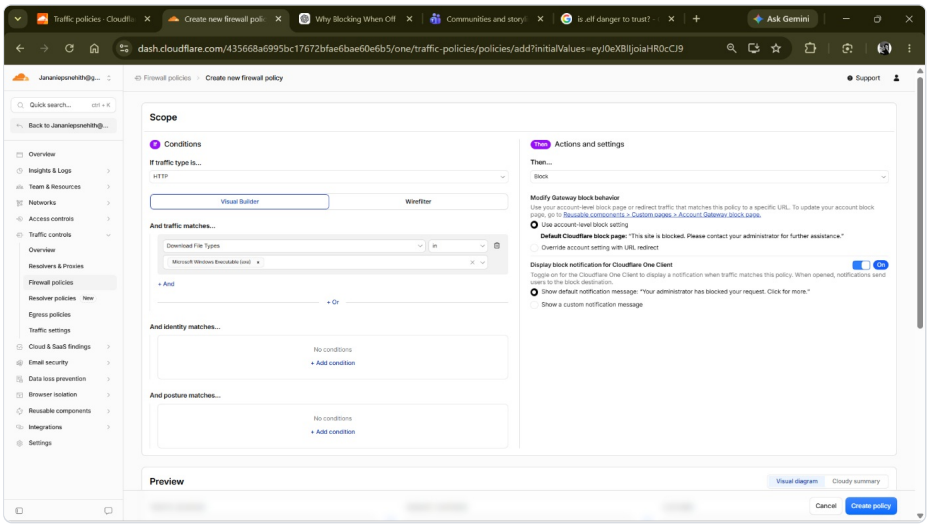
opened the **file-type selector** and selected **Microsoft Windows Executable (exe)**. This became the specific input condition for the executable-download blocking rule.



Download File Types list containing Microsoft Windows Executable (exe).

Step 12: Set the EXE Download Condition

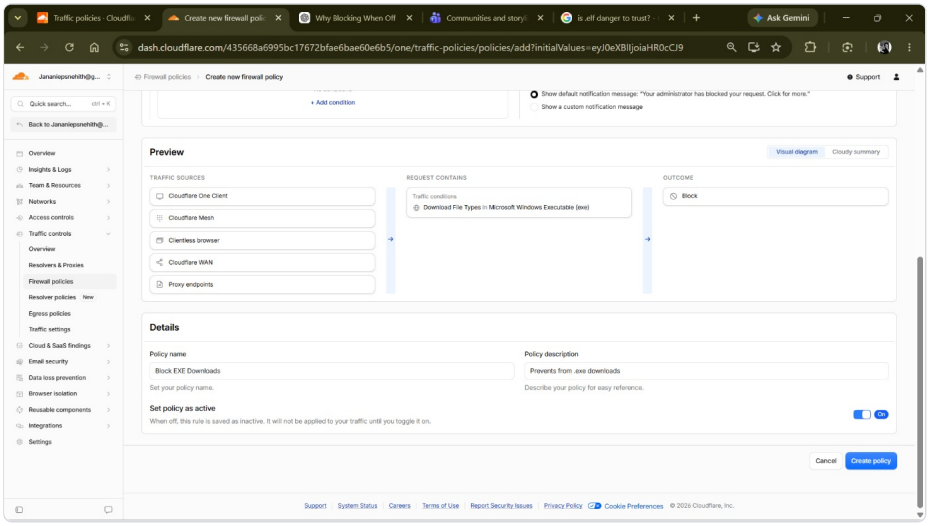
we applied the selected EXE file type as the HTTP policy condition. The rule was therefore designed to match HTTP downloads identified as Microsoft Windows executable files.



HTTP policy condition configured for Microsoft Windows Executable (exe).

Step 13: Finalize the EXE Blocking Policy

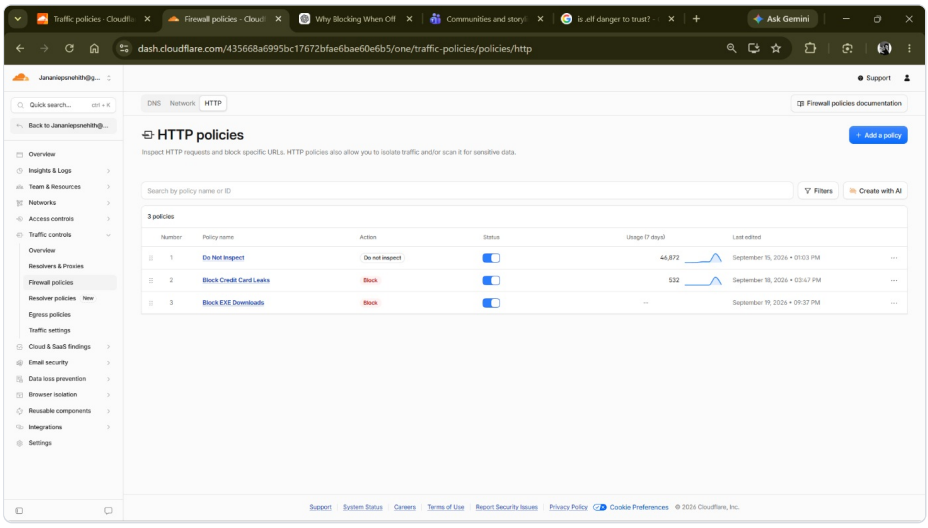
finalized the policy as **Block EXE Downloads**. The policy details show the executable file-type condition, an *active status*, and the Block outcome. This was the completed configuration for the HTTP executable-download control.



Final HTTP policy details for Block EXE Downloads, including the EXE condition and Block outcome.

Step 14: HTTP Policies Dashboard — Progress Check

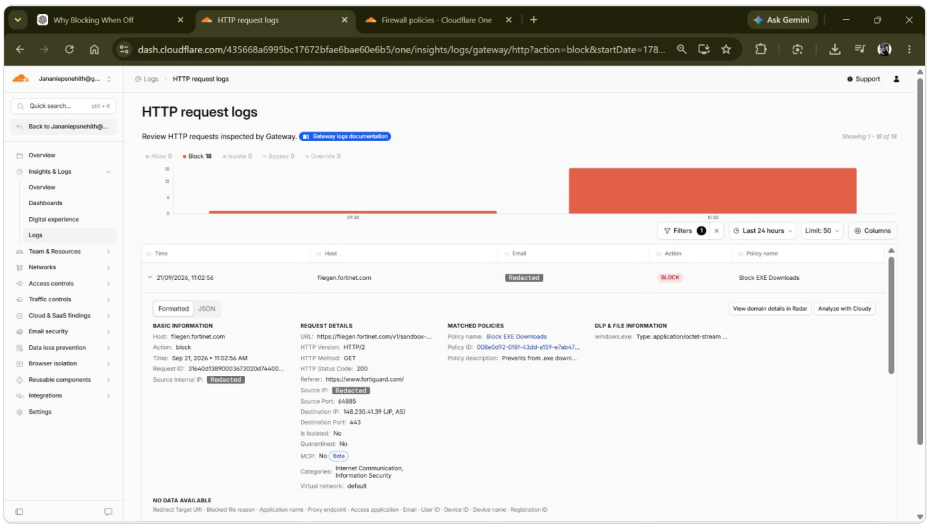
checked the HTTP Policies dashboard after saving the rule. The dashboard showed Block EXE Downloads as an active blocking policy. I used this as the HTTP configuration progress checkpoint before testing enforcement.



HTTP Policies dashboard showing the configured HTTP policies and their active states.

Step 15: HTTP Enforcement Test — EXE Download

After testing in client system, then we validated the HTTP rule through the request log. The request to **filegen.fortinet.com** was recorded with the action BLOCK and the policy **Block EXE Downloads**. The log also showed the requested URL and HTTP request details. This was the main evidence that the EXE download policy was being enforced.

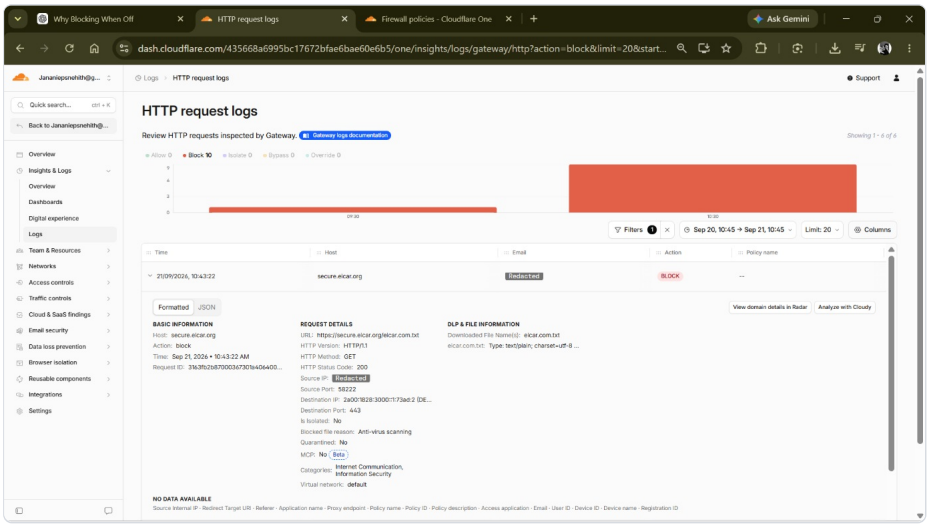


HTTP request log showing an EXE-related request blocked by Block EXE Downloads.

Result: HTTP request was recorded as BLOCK under Block EXE Downloads.

Step 16: Malware Test — EICAR

also tested access to the **EICAR anti-malware test file**. The HTTP log recorded the request as BLOCK and identified the blocked file as **eicar.com.txt**, with ***Anti-virus scanning shown as the block reason***. We treated this as separate malware-scanning evidence rather than attributing it to the EXE policy.

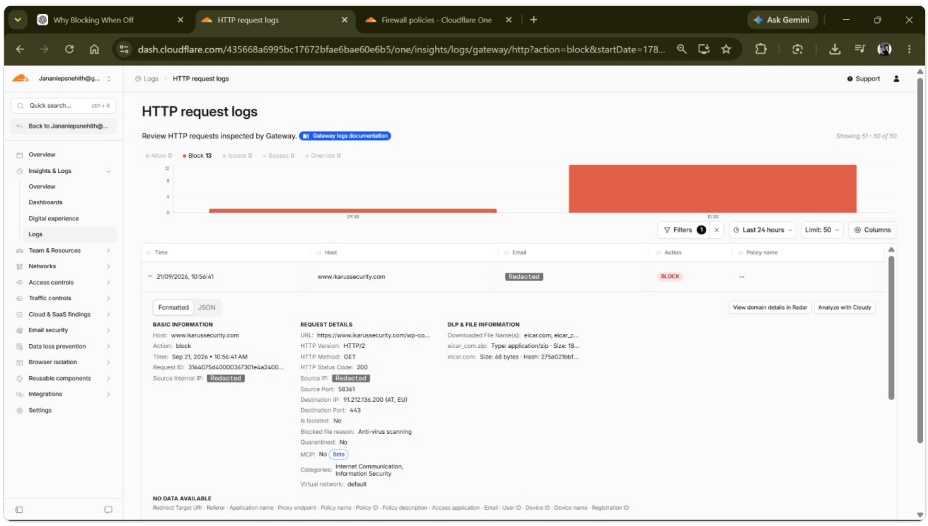


HTTP request log for the EICAR test file showing BLOCK and an anti-virus scanning block reason.

Result: EICAR test traffic was blocked by anti-virus scanning.

Step 17: Additional EICAR Blocking Result

reviewed another EICAR test result. The request was blocked and the log listed EICAR test filenames, with Anti-virus scanning shown as the blocked-file reason. This provided an additional output from the web-security controls.

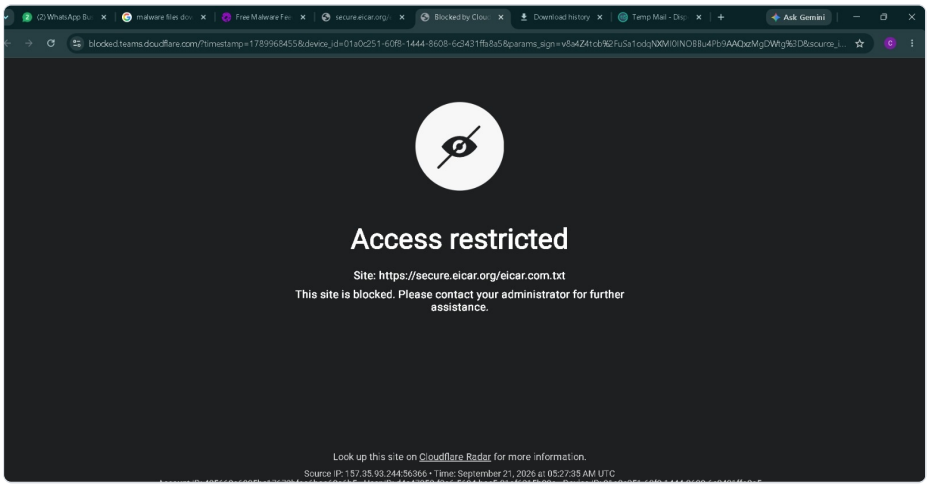


HTTP request log showing EICAR files blocked by anti-virus scanning.

Result: EICAR test traffic was blocked by anti-virus scanning.

Step 18: Browser-Level Block Confirmation

Finally, confirmed the blocking behavior from the user side in the browser. When accessed the EICAR test URL, Cloudflare displayed an Access restricted page stating that the site was blocked. This gaves a direct end-user confirmation in addition to the Gateway logs.



Browser showing Cloudflare's Access restricted page when opening the EICAR test URL.

Result: The browser displayed Cloudflare's Access restricted block page.

5. Implementation Results

Layer	Control / Test	Observed Output
DNS	Security-category filtering	Policy configured with Block outcome
DNS	Spam test	Request blocked; category shown as Spam
DNS	DGA test	Request blocked; category shown as DGA Domains
HTTP	EXE download policy	Block EXE Downloads configured and active
HTTP	EXE enforcement	Request log recorded BLOCK under Block EXE Downloads
HTTP / Security	EICAR test	Request/file blocked by Anti-virus scanning
Browser	EICAR URL access	Cloudflare Access restricted block page displayed

6. Evidence Notes and Limitations

The DNS screenshots document security-category filtering and blocked DNS requests.

The HTTP screenshots document the EXE file-type condition, the Block EXE Downloads policy, and an enforcement log.

The EICAR screenshots explicitly show Anti-virus scanning as the blocked-file reason; they are not presented as evidence that the EXE policy caused the EICAR block.

The HTTP request log for the EXE test shows Gateway Action as BLOCK; the HTTP status code shown in the log is not interpreted as successful user access.

The supplied screenshots do not provide an allow-test result, so this report does not claim an allow-path validation.

Only screenshots representing my actual policy inputs, progress, and outputs are included; guide/reference images and duplicate uploads are omitted.

7. Conclusion

We implemented and tested the Policies & SWG controls in Cloudflare Zero Trust. The evidence shows the progression

from DNS policy configuration and category selection, through DNS blocking validation, to HTTP executable-download filtering and enforcement testing. The final browser test also provided a direct view of a Cloudflare block response.

Team 6 — Data Loss Prevention (DLP)

Blocking Credit Card Leaks with Cloudflare Zero Trust Gateway

Project	**Building a Zero Trust / SASE Architecture on Cloudflare's Free Tier**
Team	Team 6 – Data
Students	Nunna Aahlada, Shaik Azaruddin
Assigned Layer	Layer 6 (Data / DLP)
Report Title	Data Loss Prevention (DLP): Blocking Credit Card Leaks via Cloudflare Gateway

1. Objective

This report documents the end-to-end implementation of a Data Loss Prevention (DLP) control on Cloudflare's Zero Trust Gateway, assigned to Team 6 as Layer 6 (Data) of the broader Zero Trust / SASE architecture built on Cloudflare's free tier. The goal was to detect and block credit card numbers leaving the network through HTTP/HTTPS traffic, using Cloudflare's predefined DLP profiles combined with an HTTP Gateway policy.

2. Step 1 — Enable HTTPS Traffic Inspection

Because DLP inspects payload content in transit, HTTPS decryption had to be enabled on the test device first. This was configured under Gateway → Traffic policies → Traffic settings, where the "Inspect HTTPS requests with TLS decryption" toggle was switched On, allowing Cloudflare to decrypt, inspect, and re-encrypt HTTPS traffic for DLP matching.

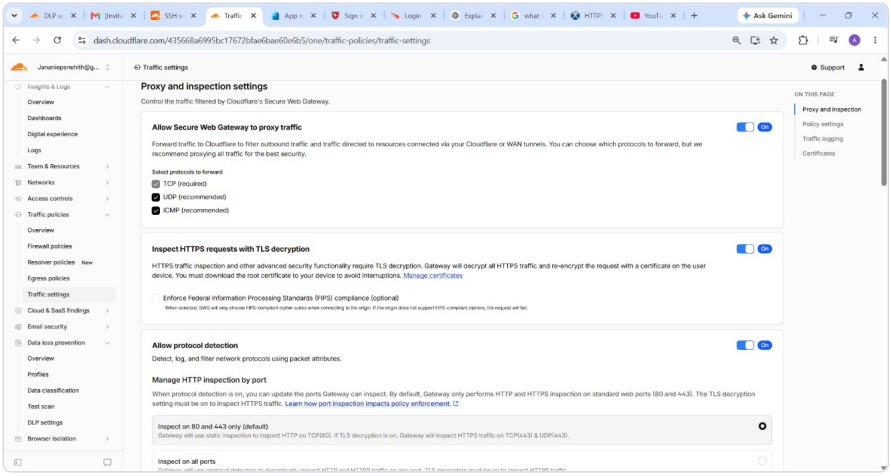


Figure 1: Proxy and inspection settings — HTTPS traffic inspection (TLS decryption) enabled.

With TLS decryption enabled, Cloudflare requires the device to trust its Gateway root certificate; otherwise browsers reject the re-signed certificate. The certificate was generated and downloaded from Traffic settings → Certificates.

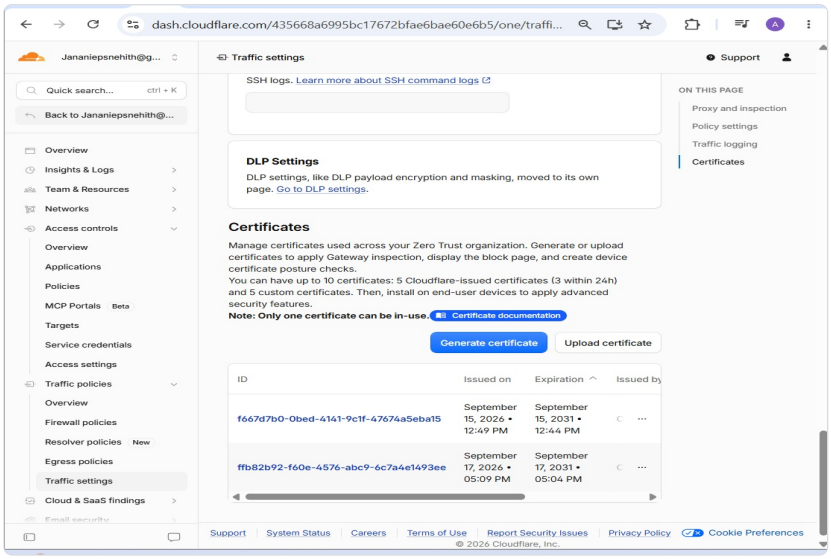


Figure 2: Traffic settings — Certificates section listing the Gateway CA certificates.

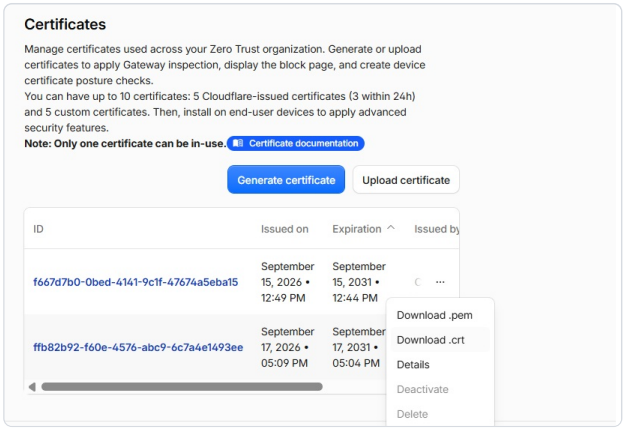


Figure 3: Downloading the Gateway CA certificate (.crt / .pem).

The downloaded certificate was then installed into the local trust store using the Windows Certificate Import Wizard, placing it explicitly under Trusted Root Certification Authorities so the browser would trust Cloudflare's re-signed HTTPS traffic.

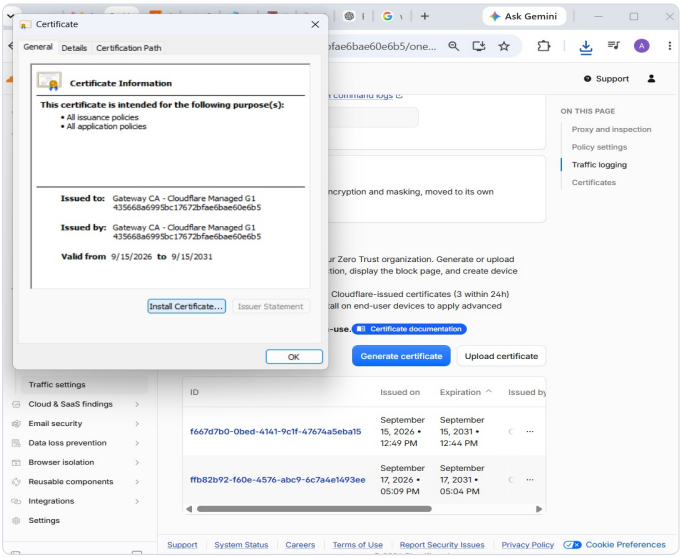


Figure 4: Certificate details dialog before installation.

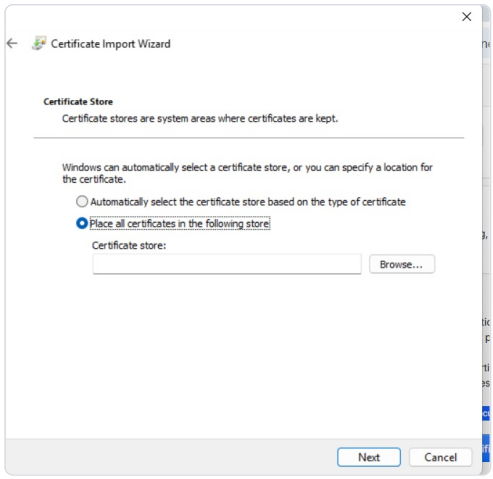


Figure 5: Certificate Import Wizard — selecting a certificate store location.

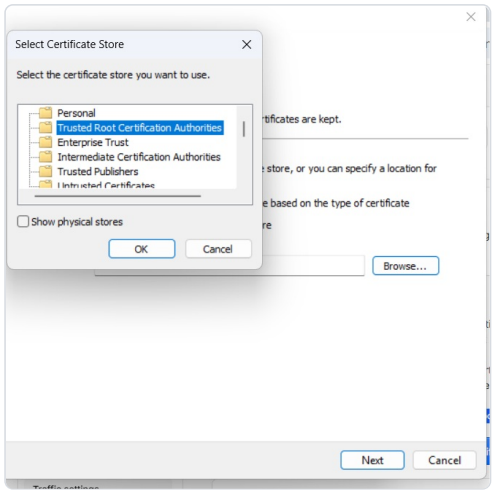


Figure 6: Selecting "Trusted Root Certification Authorities" as the target store.

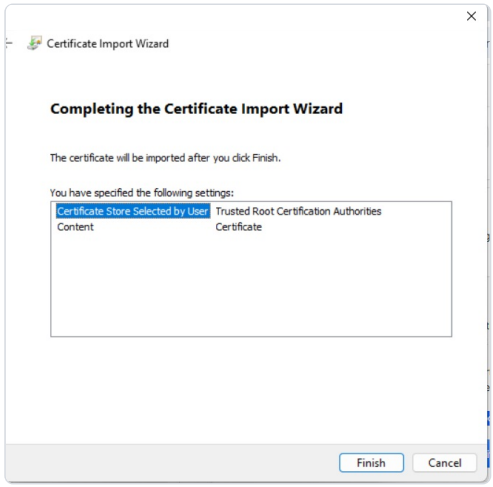


Figure 7: Completing the Certificate Import Wizard.

3. Step 2 — Configure the DLP Profile

In the Zero Trust dashboard, the built-in predefined DLP profiles were reviewed under Gateway → Data loss prevention → Profiles. Cloudflare's free tier groups credit-card detection under the predefined "Financial Information" profile (alongside a separate "Social Security, Insurance, Tax, and Identifier Numbers" profile).

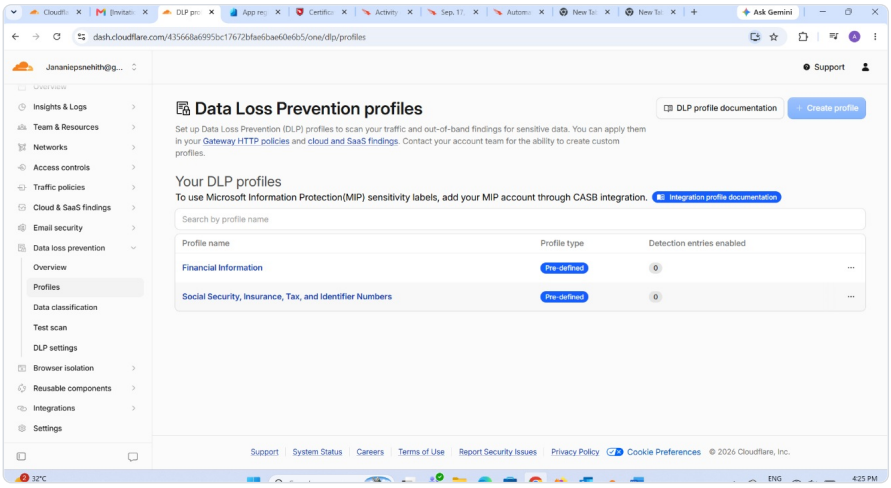


Figure 8: DLP profiles list, showing the predefined "Financial Information" profile used for credit card detection.

Inside the Financial Information profile, the specific detection entries were enabled, including American Express, Visa, Mastercard, Diners Club, and CVV card-number patterns, before saving the profile so it would actively scan for these patterns.

Under DLP settings, related account-level options — OCR-based image scanning, AI context analysis, and payload log masking — were also reviewed to understand how matched payloads would be logged.

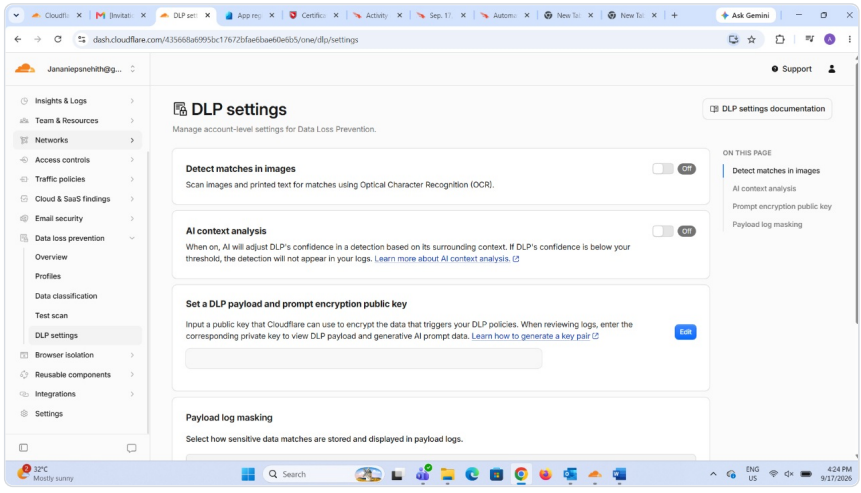


Figure 9: Account-level DLP settings (image scanning, AI context analysis, payload log masking).

4. Step 3 — Create the HTTP Gateway DLP Policy

A new rule, "Block Credit Card Leaks," was created under Gateway → Firewall policies → HTTP, with the DLP profile from Step 2 set as the matching condition and the action set to Block.

The first version of the policy used a broad catch-all condition (URL matches regex .*) combined with the DLP profile condition.

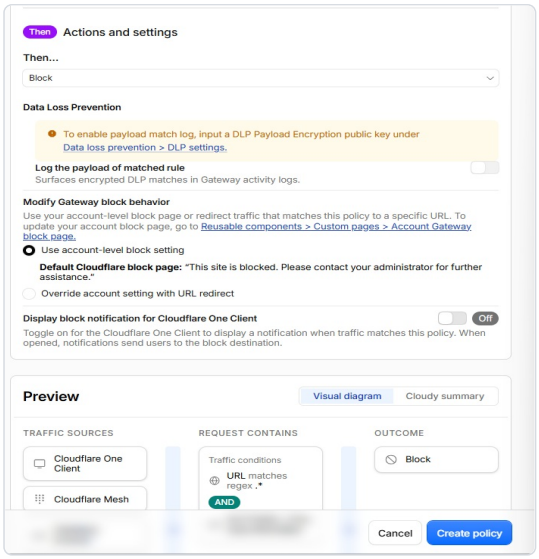


Figure 10: Initial policy draft — traffic condition set to match all URLs, combined with the Financial Information DLP profile, action set to Block.

Troubleshooting: refining the match condition

During testing, this configuration caused two issues that had to be diagnosed and corrected:

1. The two conditions were initially evaluated with an OR relationship rather than AND, which caused the policy to block all traffic on the network — including unrelated sites such as YouTube and Telegram — regardless of whether a credit card number was present.
2. Even after correcting the logic to AND, using a wildcard URL match (.*) against all traffic combined with a DLP profile check still over-matched ordinary browsing traffic.

The fix was to replace the broad URL wildcard condition with a narrower HTTP Method condition (POST), since credit card data can only be leaked through data submissions such as form posts or file uploads — not simple page views (GET requests). The final policy logic combined "HTTP Method in POST" AND "DLP Profile in Financial Information," restoring normal browsing while still catching sensitive data submissions.

Condition 1	**DLP Profile in Financial Information**
Condition 2	HTTP Method in POST
Logical Operator	AND
Action	Block
Traffic Type	HTTP

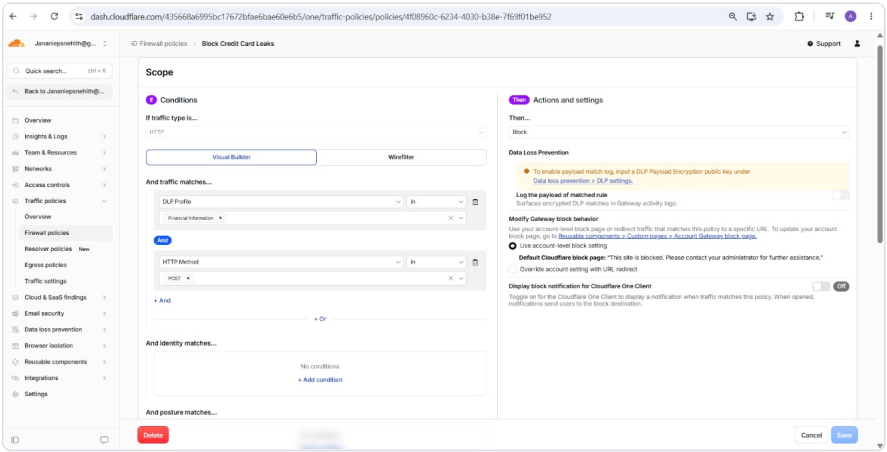


Figure 11: Final HTTP policy "Block Credit Card Leaks" — conditions (DLP Profile AND HTTP Method: POST) and action (Block).

5. Step 4 — Test and Trigger the Policy

With the WARP client connected and routing the test device's traffic through the Cloudflare Gateway, the policy was tested using dlptest.com, a public utility site built for safely testing DLP controls. A dummy, non-real Visa test number (4111 1111 1111 1111) was submitted through the site's HTTPS POST test form.

An early test attempt over plain HTTP did not trigger the policy, since unencrypted HTTP traffic bypasses the TLS decryption engine that DLP inspection depends on; the test was repeated over HTTPS, which correctly triggered the Gateway policy.

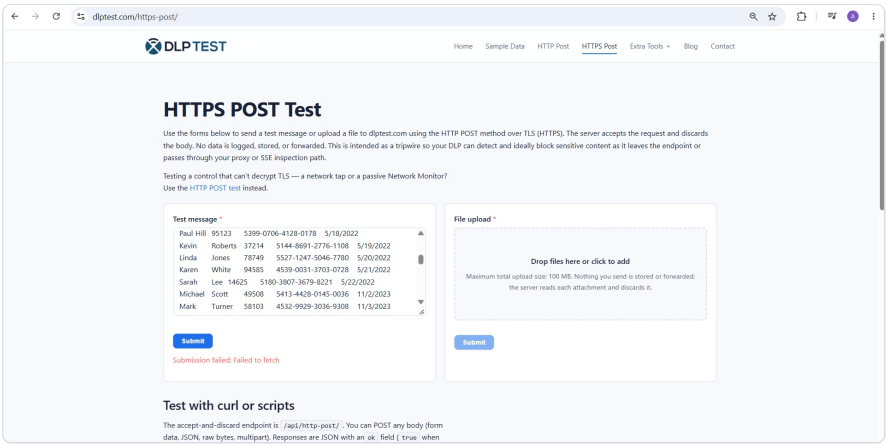


Figure 12: HTTPS POST test on dlptest.com — the submission was intercepted and failed to reach the destination, confirming the DLP policy blocked the request containing the test credit card pattern.

6. Step 5 — Gather Evidence and Analytics

Following the successful block, the Gateway → Analytics / Logs section was used to locate the corresponding HTTP request log entry for the test submission to dlptest.com, confirming the Action as Block and the matching rule as "Block Credit Card Leaks," with the detection attributed to the credit card pattern match from the Financial Information DLP profile. Screenshots of both the DLP policy configuration and the triggered log entry were collected as evidence for this report, in line with the task requirements.

7. Summary

Team 6 successfully implemented the Layer 6 (Data / DLP) control of the Zero Trust / SASE architecture on Cloudflare's free tier. The final working configuration:

- Enables HTTPS (TLS) inspection on the Gateway, with the Cloudflare root certificate trusted on the test device.
- Enables the predefined "Financial Information" DLP profile, with credit card detection entries turned on.
- Applies an HTTP Gateway policy that blocks any POST request matching the Financial Information DLP profile.
- Was validated end-to-end using a safe dummy credit card number on dlptest.com, confirming the request was blocked over HTTPS while normal browsing traffic remained unaffected.

DIGITAL EXPERIENCE MONITORING AND NETWORK PERFORMANCE MONITORING USING CLOUDFLARE ZERO TRUST

Team 7 — Visibility: DEX and Network Flow Monitoring

1. Objective

To monitor the availability and performance of the application published through Cloudflare Zero Trust using HTTP synthetic testing and network latency monitoring, and to analyze the collected results to identify connectivity and performance issues.

2. Scope

Component	What is monitored
Application	Application published by Team 4
HTTP Synthetic Test	Application availability and HTTP response performance
Latency Test	Network response time to the target
Network Analytics	Traffic patterns and network activity

3. Prerequisites

Access to the Cloudflare Zero Trust dashboard.

The application published by Team 4 is available for testing.

The application URL is available: dash.demotest.space.

DEX testing is available in the Cloudflare environment.

The test target is reachable and correctly configured.

4. Test 1 – HTTP Synthetic Monitoring

4.1 Purpose

To verify whether the application is reachable and responding to HTTP requests, while collecting response-time and availability metrics.

4.2 Test Configuration

Parameter	Configured Value
Test Type	HTTP GET
Target URL	dash.demotest.space
Test Frequency	5 minutes
Devices Targeted	10
DEX Rules	All source rules
Monitoring Platform	Cloudflare Zero Trust DEX

4.3 Procedure

Open the Cloudflare Zero Trust Dashboard.

Navigate to Digital Experience Monitoring (DEX) → Tests.

Create or open the HTTP synthetic test.

Select HTTP GET as the test type.

Enter the application target: dash.demotest.space.

Set the test frequency to 5 minutes.

Configure the applicable devices and DEX rules.

Enable the test and allow it to collect monitoring data.

Open the test results page and review the performance graphs.

Capture screenshots of the configuration and the collected results.

4.4 Test Results and Graphs

The HTTP GET synthetic test results were reviewed to evaluate application response performance and availability. The dashboard displayed resource fetch time, server response time, DNS response time, availability, and HTTP status code information.

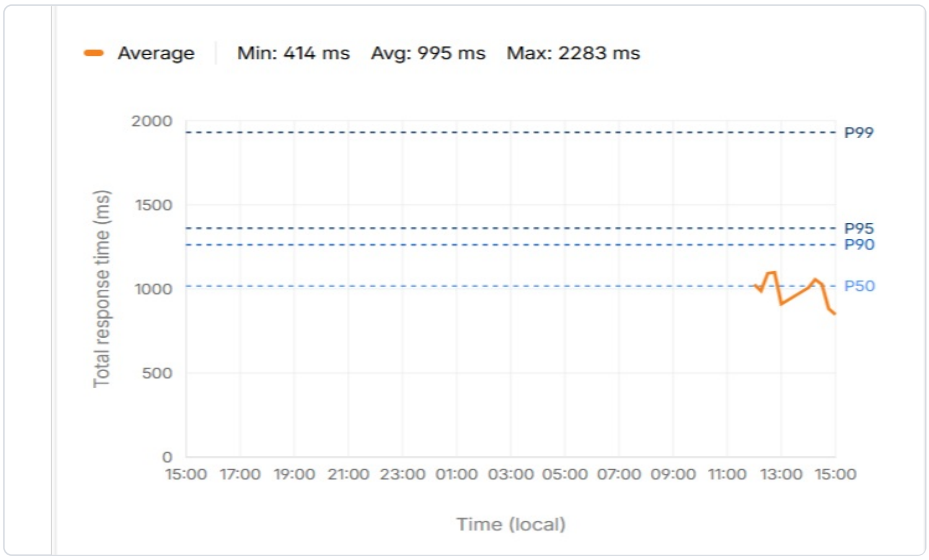


Figure 1: Resource Fetch Time

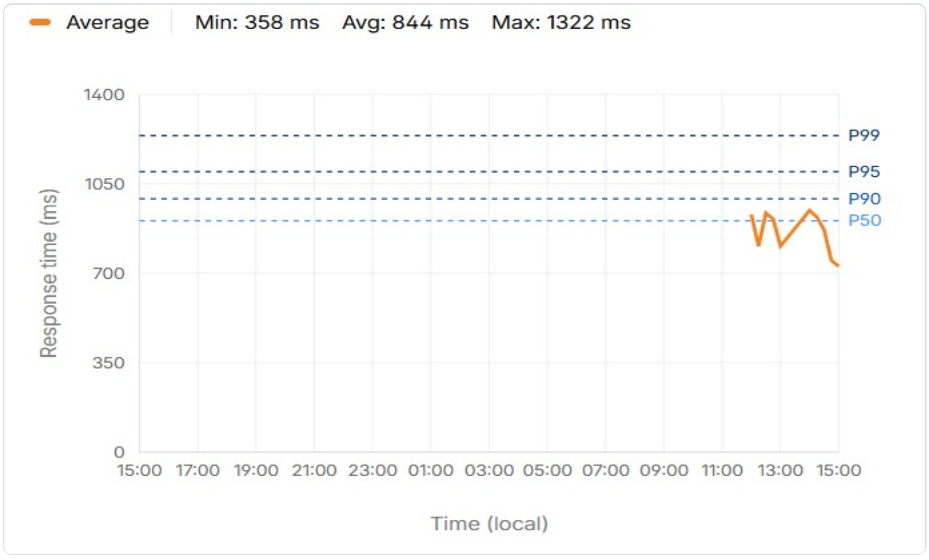


Figure 2: Server Response Time

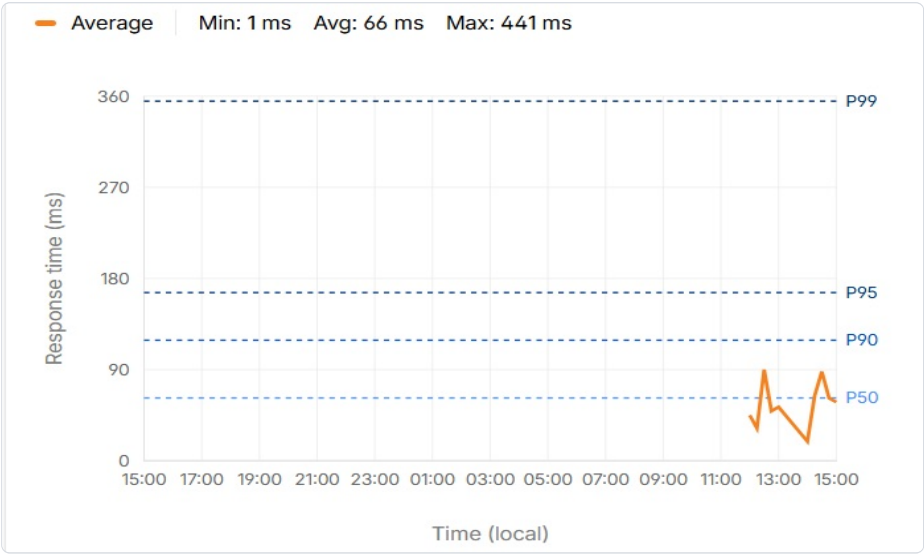


Figure 3: DNS Response Time

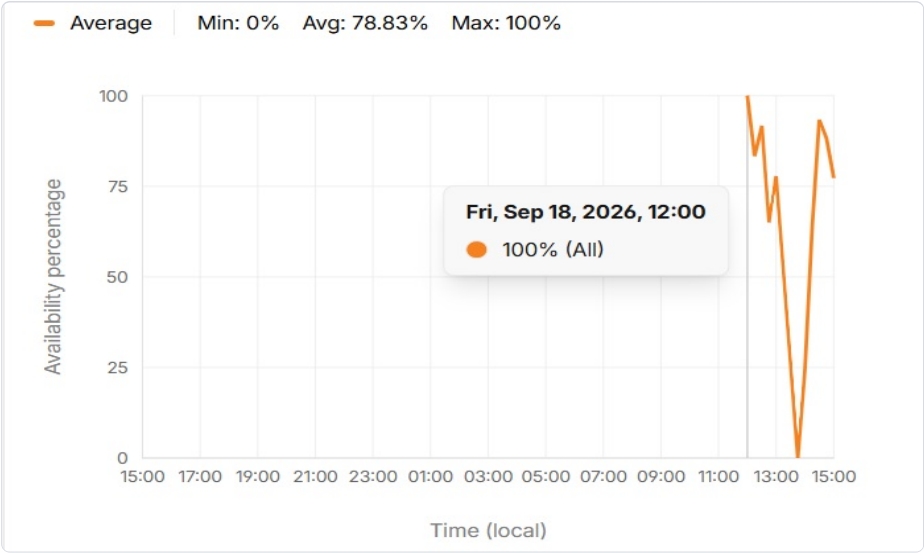


Figure 4: Availability

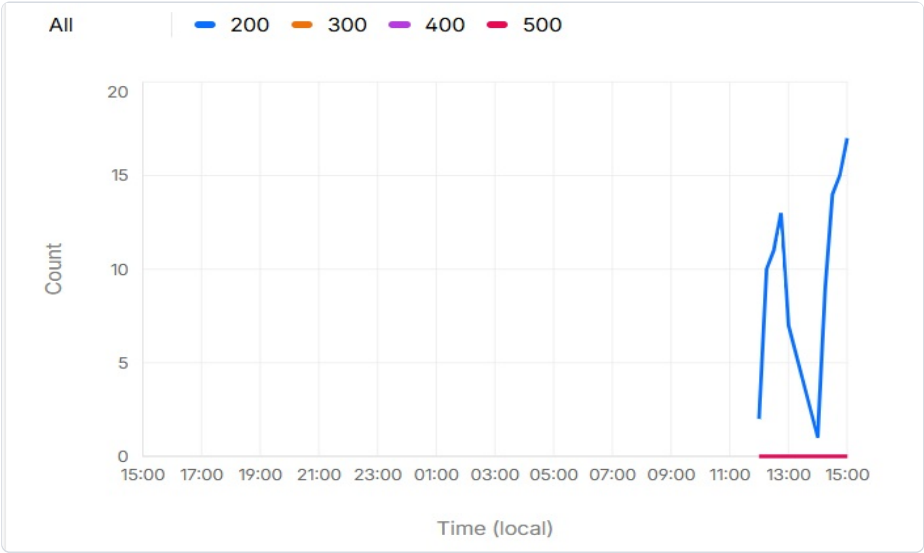


Figure 5: HTTP Status Codes

4.5 Observed Results

The following values were recorded from the displayed Cloudflare DEX HTTP GET test results:

Metric	Minimum	Average	Maximum
Resource Fetch Time	414 ms	995 ms	2283 ms
Server Response Time	358 ms	844 ms	1322 ms
DNS Response Time	1 ms	66 ms	441 ms
Availability	0%	78.83%	100%

4.6 Result Analysis

The HTTP GET synthetic test was configured for dash.demotest.space with a five-minute test frequency. The test recorded an average resource fetch time of 995 ms, which includes DNS resolution and server response time. The average server response time was 844 ms, while the average DNS response time was 66 ms. The displayed average availability was 78.83%. The difference between the minimum and maximum response times indicates variation in application response performance. Continued monitoring is required to determine whether the observed availability and latency variations are persistent.

5. Test 2 – Network Latency Monitoring

5.1 Purpose

To measure the time taken for network communication between the monitoring location and the target, and to identify latency variations over time.

5.2 Test Configuration

Parameter	Configured Value
Test Name	Network_Latency
Target	dash.demotest.space
Test Type	Traceroute
Test Frequency	5 minutes
Devices Targeted	10
DEX Rules	All source rules
Monitoring Platform	Cloudflare Zero Trust DEX
Time Range	Past 24 Hours

5.3 Procedure

Navigate to DEX → Tests.

Create a network test, if the option is available in the plan.

Select the latency or suitable network test type.

Enter the target associated with the monitored application.

Configure the test interval and enable the test.

Allow the test to collect results over time.

Review the latency graph and record the minimum, average, and maximum values.

Capture a screenshot of the latency results.

5.4 Test Results and Graphs

The Network_Latency test was configured as a traceroute test for dash.demotest.space with a five-minute test frequency. The test targeted 10 devices and displayed network performance metrics for the selected past 24-hour period.

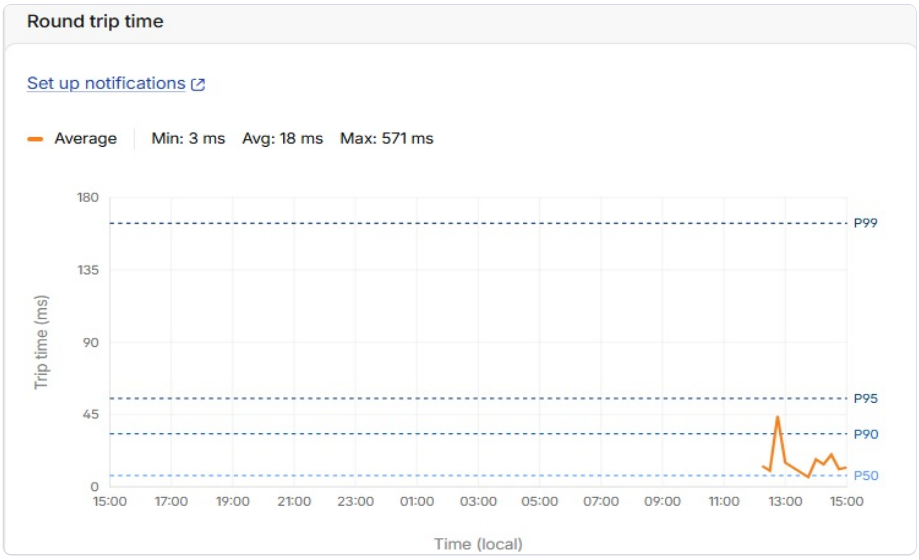


Figure 6: Round Trip Time

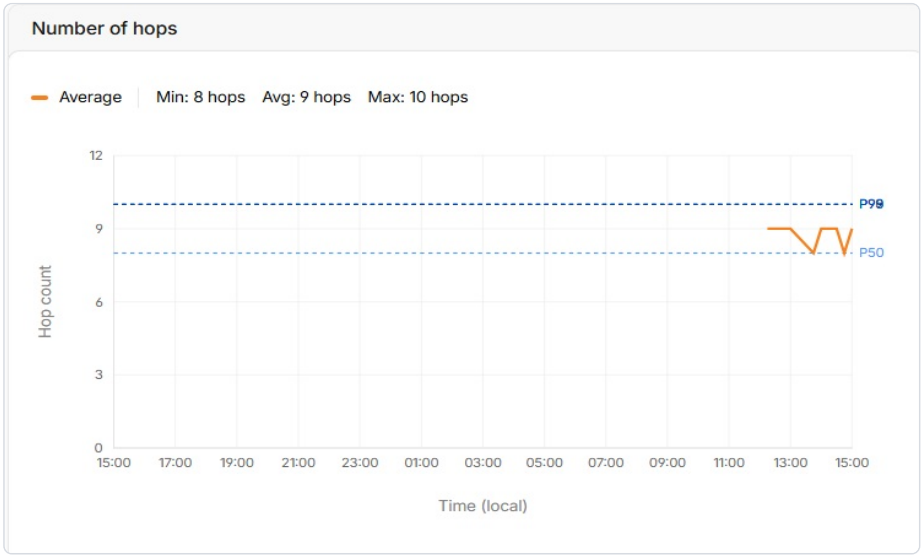


Figure 7: Number of Hops

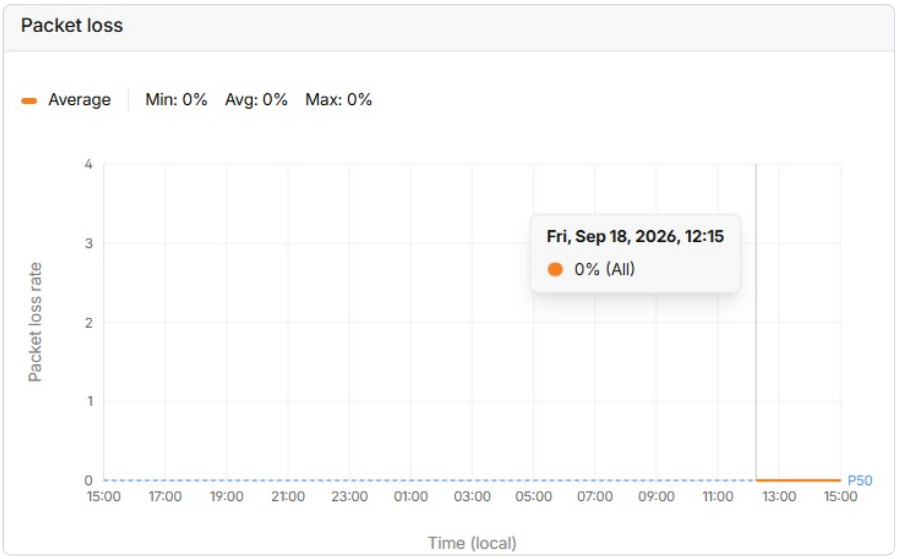


Figure 8: Packet Loss

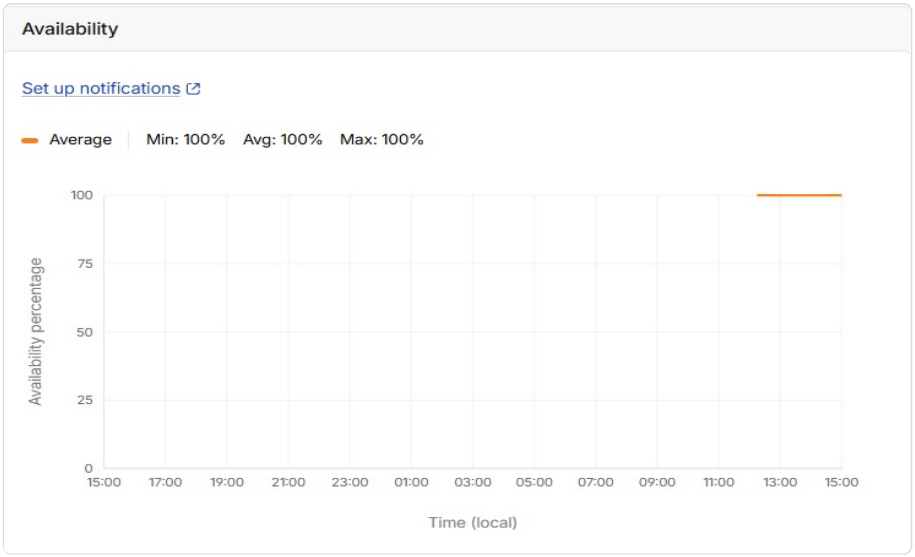


Figure 9: Availability

5.5 Observed Results

The following values were recorded from the displayed Cloudflare DEX Network_Latency test results:

Metric	Minimum	Average	Maximum
Round Trip Time	3 ms	18 ms	571 ms
Number of Hops	8 hops	9 hops	10 hops
Packet Loss	0%	0%	0%
Availability	100%	100%	100%

5.6 Result Analysis

The traceroute-based network latency test recorded an average round trip time of 18 ms, with a minimum of 3 ms and a maximum of 571 ms. The route contained an average of 9 hops, ranging from 8 to 10 hops. Packet loss remained at 0%, and the recorded availability was 100% during the displayed monitoring period. The difference between the minimum and maximum round trip times indicates variation in latency during the observation period.

6. Network Analytics and Flow Monitoring

6.1 Purpose

To observe overall network traffic activity and identify traffic patterns during the monitoring period.

6.2 Procedure

Open the Cloudflare Zero Trust Dashboard.

Navigate to Gateway → Analytics → Network.

Select an appropriate time range covering the test period.

Review the available traffic volume, protocol, application, source, or destination information.

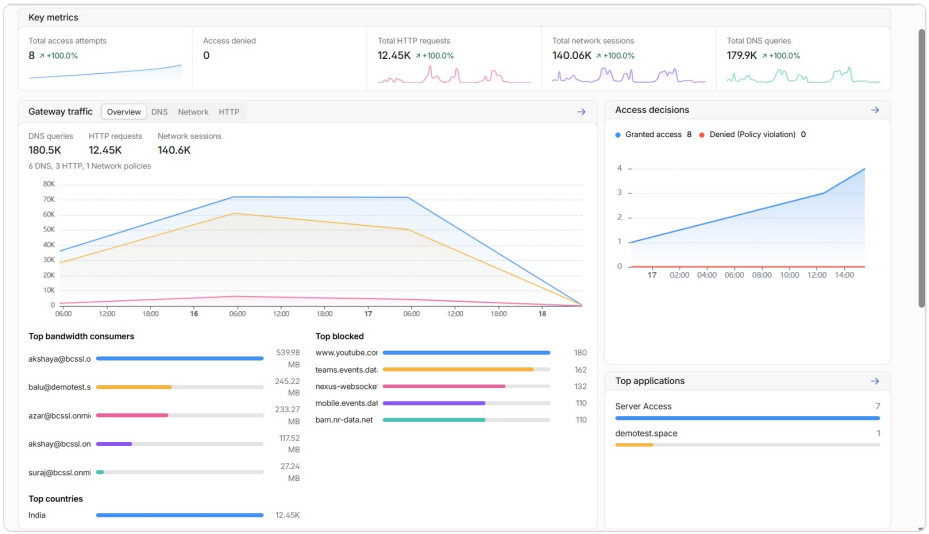
Observe whether traffic volume changes during application or web-filtering tests.

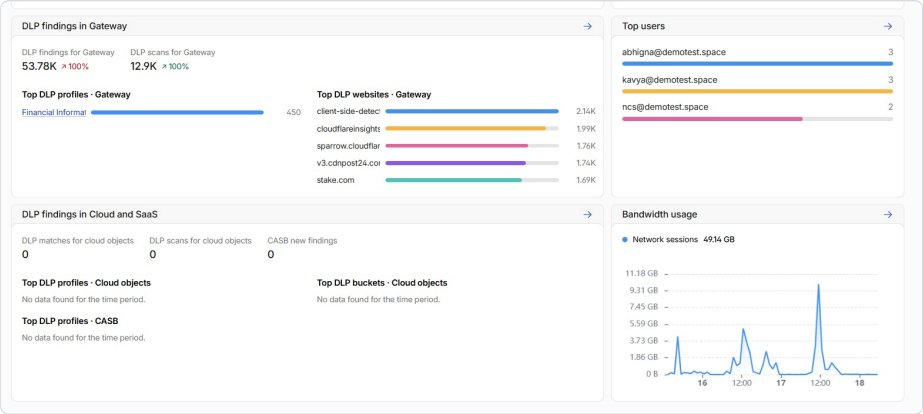
Capture a screenshot of the network analytics view.

6.3 Network Analytics Observations

The following fields should be completed after reviewing the Gateway network analytics dashboard. Only observations visible in the dashboard should be recorded.

Observation Parameter	Recorded Observation	
Selected time range | Displayed monitoring period | | covering 16–18 September		
Total access attempts	8	
Total HTTP requests	12.45K	
Total network sessions	140.06K	
Total DNS queries	179.9K	





6.4 Network Analytics Analysis

The Cloudflare Gateway Network Analytics dashboard provided visibility into network traffic and access activity during the selected monitoring period. The dashboard recorded 8 total access attempts, with 8 granted and 0 denied access decisions. It also displayed 12.45K HTTP requests, 140.06K network sessions, and 179.9K total DNS queries in the key metrics section. The traffic overview showed DNS queries, HTTP requests, and network sessions over time, while the bandwidth and application panels provided additional visibility into traffic consumption and application activity. The displayed top country was India, and the Top Applications panel showed Server Access and demotest.space. These observations demonstrate how Gateway Analytics can be used to monitor traffic patterns, access decisions, and network activity.

7. Overall Findings

The Cloudflare DEX HTTP GET and Network Latency tests were configured for dash.demotest.space with a five-minute test frequency. The HTTP test provided visibility into application response performance, DNS resolution, and availability. The observed average resource fetch time was 995 ms, average server response time was 844 ms, average DNS response time was 66 ms, and average availability was 78.83%.

The Network_Latency traceroute test recorded an average round trip time of 18 ms, an average of 9 network hops, 0% packet loss, and 100% availability during the displayed monitoring period.

The Cloudflare Gateway Network Analytics dashboard provided visibility into network traffic and access activity. It displayed 8 total access attempts, 8 granted access decisions, 0 denied access decisions, 12.45K HTTP requests, 140.06K network sessions, and 179.9K DNS queries. The dashboard also showed traffic trends, bandwidth consumers, blocked destinations, and top applications.

Together, these monitoring activities provide visibility into application performance, network connectivity, access activity, and traffic patterns within the Cloudflare Zero Trust environment.

The Cloudflare DEX HTTP GET and Network_Latency tests were configured for dash.demotest.space with a five-minute test frequency. The HTTP test provided visibility into application response performance, DNS resolution, and availability. The observed average resource fetch time was 995 ms, average server response time was 844 ms, average DNS response time was 66 ms, and average availability was 78.83%.

The Network_Latency traceroute test recorded an average round trip time of 18 ms, an average of 9 network hops, 0% packet loss, and 100% availability during the displayed monitoring period.

The Cloudflare Gateway Network Analytics dashboard provided visibility into network traffic and access activity. It displayed 8 total access attempts, 8 granted access decisions, 0 denied access decisions, 12.45K HTTP requests, 140.06K network sessions, and 179.9K DNS queries. The dashboard also showed traffic trends, bandwidth consumers, blocked destinations, and top applications.

Together, these monitoring activities provide visibility into application performance, network connectivity, access activity, and traffic patterns within the Cloudflare Zero Trust environment.

8. Conclusion

Digital Experience Monitoring provides visibility into application availability and performance. The HTTP synthetic test helps verify whether the application responds to HTTP requests, while latency monitoring helps evaluate network

response time, route characteristics, packet loss, and availability. Network analytics adds visibility into traffic patterns. Together, these monitoring activities support the identification of performance and connectivity issues in the Zero Trust environment.

9. Reference

BCSS Academia – SNIST Capstone: Team Step-by-Step Guide, Team 7 – Visibility (DEX & Network Flow Monitoring).